

中国研究生创新实践系列大赛
“华为杯”第二十二届中国研究生
数学建模竞赛

题 目 围岩裂隙精准识别与三维模型重构

摘 要:

煤炭资源作为我国能源安全的核心支柱，其高效且安全开采却一直受到巷道围岩裂隙所引发的冒顶、突水等事故的严重制约。传统的岩芯取样方法存在耗时较长、成本高昂、覆盖范围有限等不足，而钻孔成像技术则可依托现有锚杆支护钻孔，通过 360° 全景扫描实现孔壁的可视化观察，但仍面临干扰因素复杂、人工判读效率低下、三维重构精度不足等挑战。本文针对钻孔成像技术在实际工程中的应用难点，围绕“裂隙识别-定量表征-连通分析-三维重构”四个核心研究环节为主线，结合煤系地层实测数据开展系统研究。

针对问题一，建立**钻孔裂隙自动判别模型**。本文首先利用数据增强策略避免数据稀少与过拟合的现象，接着，利用涵盖灰度化、光照矫正以及非局部均值一系列图像预处理操作进一步优化图像。通过建立三种裂隙决策算法，即基于多特征融合的 K-Means 聚类融合和 Canny 边缘检测模型，以及 YOLOv8 模型分别对表面裂缝检测，方案对比后，选择**多特征融合的 Canny 边缘检测模型**，进一步通过后处理优化决策优化结果，最终检测裂隙数量分别为**1 条 (图 1-1)、6 条 (图 1-2)、3 条 (图 1-3)**。

针对问题二，构建“**正弦状**”裂隙**自动聚类与参数反演模型**。本文首先利用问题 1 中的 Canny 边缘检测及形态学处理，得到二值化边缘图像。接着，基于空间接近性、方向一致性一致性准则以及 **GMM 聚类**算法对边缘像素进行聚类，过滤小噪声聚类后提取各裂隙中心线点集。通过构建“正弦状”裂隙数学模型，采用梯度下降法-牛顿法融合及神经网络的参数回归优化算法，最小化残差平方和以求解振幅 **R**、残差 **P** 等参数。用决定系数 R^2 分别对比拟合效果，神经网络拟合参数能有效实现裂隙的定量表征。最后将反演的附件 2 中图 2-1 的**5 条**、图 2-2 的**1 条**和图 2-3 的**3 条**裂隙参数填入表中。

针对问题三，本文建立了基于正弦框架的裂隙粗糙度综合评价体系。基于前面方法对复杂裂隙进行表征，成功识别出裂隙条数分别为**1 (图 3-1)、1 (图 3-2)、6 (图 3-3)**，提取裂隙轮廓并离散化，用等间距采样计算出 JRC 值 (**33.12, 33.48, 38.01, 35.82, 30.18, 36.49, 28.96**)。对不同采样方法进行比较，提出曲率-紧凑度加权的自适应采样策略，实现了根据**轮廓复杂度动态非均匀优化采样密度**。研究发现，裂隙面积对 JRC 值呈弱正相关，

传统模型对此不敏感。构建**二阶导数的 JRC 改进模型**，有效克服了一阶导数对线性趋势敏感的局限性。

针对问题四，构建**多钻孔裂隙网络三维重构与不确定性评估模型**。首先，基于正弦曲线参数反演三维裂隙平面产状（倾角、倾向），建立平面方程。其次，提出包含空间距离贴近度、产状相似度、JRC 粗糙度相似度及垂向深度重叠度的多因子加权综合评价模型，计算相邻钻孔间裂隙的连通概率。在此基础上，对研究区域进行三维网格离散化，利用信息熵量化每个网格单元的不确定性，识别出高不确定性区域。最后，以最大化**信息增益**为目标，兼顾工程约束（最小间距 $\geq 500\text{mm}$ ，孔深 5000mm ），推荐了 3 个最优补充钻孔位置 **(1100, 1500, 0)**、**(1900, 1500, 0)**、**(1900, 1100, 0)**（单位：mm），并确定其优先级，为后续勘探布设提供决策依据。

本文通过系统研究，构建了从钻孔图像智能识别到三维裂隙网络重构的完整技术框架，有效提升了巷道围岩裂隙分析的自动化与精准化水平。研究成果为煤矿安全高效开采提供了重要的技术支撑，推进了钻孔成像技术在岩体裂隙定量表征与风险评估领域的应用研究。未来可进一步结合多源监测数据与地质力学模型，实现围岩稳定性的动态智能预警。

关键词： 多特征融合 Canny 边缘检测模型；梯度下降法-牛顿法融合模型；神经网络的参数反演模型；基于二阶导数的 JRC 改进模型；三维裂隙平面模型；连通概率；信息熵

目录

1. 问题背景与问题重述	6
1.1 问题背景	6
1.2 问题重述	6
2. 模型假设及关键性符号说明	7
2.1 模型假设	7
2.2 符号说明	8
3. 问题一的模型建立与求解	9
3.1 问题一分析	9
3.2 基于小样本的数据增强策略	10
3.3 图像预处理	10
3.4 裂隙判别决策模型	13
3.4.1 多特征融合的 K-Means 聚类融合算法	13
3.4.2 多特征融合的 Canny 边缘检测	15
3.4.3 YOLOv8 表面裂隙检测	16
3.5 后处理优化决策 (Post-Processing)	17
3.6 方案对比与结果优化	18
3.6.1 方案对比	18
3.6.2 结果优化	18
3.7 问题小结	19
4. 问题二的模型建立与求解	20
4.1 问题二分析	20
4.2 模型准备	20
4.2.1 图像准备	20
4.2.2 裂隙聚类分析	21
4.2.3 GMM 聚类算法	21
4.3 建立“正弦”参数反演模型	23
4.4 模型求解	23
4.4.1 基于梯度下降-牛顿法混合优化求解模型	23
4.4.2 基于神经网络的参数回归优化模型	26

4.5 模型拟合对比评估.....	29
4.6 问题二结果展示与小结.....	30
5. 问题三的模型建立与求解	32
5.1 问题三分析.....	32
5.2 复杂裂隙数据表征.....	33
5.3 基于轮廓点的 JRC 计算模型	33
5.3.1 裂隙轮廓提取	33
5.3.2 边界跟踪算法离散化	34
5.3.3 等间距采样的 JRC 计算模型及求解.....	35
5.4 不同采样方法对 JRC 影响	35
5.4.1 离散坐标选取影响分析	35
5.4.2 离散点选取优化策略	37
5.5 基于二阶导数的 JRC 改进模型	38
5.5.1 裂隙面积对粗糙度评价分析	38
5.5.2 传统 JRC 模型的局限性分析.....	40
5.5.3 改进 JRC 模型的建立与求解.....	40
5.6 问题小结.....	41
6. 问题四的模型建立与求解	42
6.1 问题四分析.....	42
6.2 三维裂隙平面模型重构.....	42
6.3 多钻孔裂隙网络的连通性分析.....	42
6.3.1 平面产状与正弦曲线参数的几何关系	42
6.3.2 三维平面方程的推导	43
6.3.3 多钻孔裂隙的连通概率模型计算	43
6.4 多钻孔裂隙网络的三维重构.....	44
6.4.1 基于信息熵的不确定性区域识别模型	44
6.4.2 钻孔优化布设模型的补充	45
6.5 实验数据与参数设置.....	45
6.6 结果展示与三维可视化.....	45
6.6.1 裂隙连通概率分析结果	45
6.6.2 钻孔连通性分析结果	46
6.6.3 三维空间结构分析结果	46
6.6.4 高不确定性区域分析结果	47
6.7 问题小结.....	48

7. 模型总结	49
7.1 模型的评价.....	49
7.2 模型的改进.....	49
7.3 模型的推广.....	50
参考文献.....	51
附录 A 问题一主要代码（代码由 AI 辅助编程）.....	52
附录 B 问题二主要代码（代码由 AI 辅助编程）.....	53
附录 C 问题三主要代码（代码由 AI 辅助编程）.....	60
附录 D 问题四主要代码（代码由 AI 辅助编程）.....	64

1. 问题背景与问题重述

1.1 问题背景

煤炭是我国的主要能源，国家特别注重发展新质生产力实现煤矿安全、高效、智能化开采。井工煤矿开采是一个复杂的地下工程系统（如图 1 所示），其中巷道作为煤炭生产的咽喉和必要工程，承担着井下通风、材料及煤炭运输等关键功能。然而，巷道围岩内部肉眼难见的裂隙网络 [1, 2]，常因探测不清、不准，不及时控制而引发冒顶、突水、瓦斯突出等安全事故，严重威胁生命财产安全。因此，定量分析裂隙的几何特征 [3]，对煤矿安全生产具有重要意义。

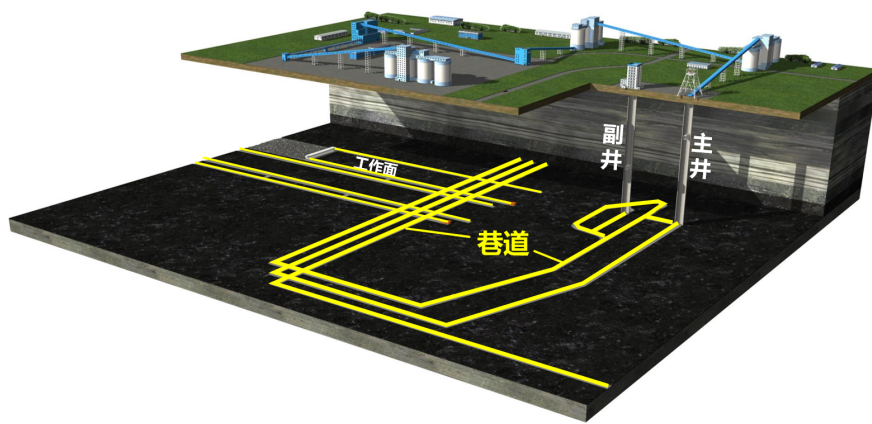


图 1 井工煤矿开采示意图

长期以来，围岩裂隙探测依赖岩芯取样，虽直观可靠，但耗时长、成本高、劳动强度大，无法实时大范围连续探测，难以满足现代煤矿对数据的高要求。近年，钻孔成像技术崛起：利用锚杆支护钻孔，通过高清摄像头 360° 扫描孔壁，将柱面孔壁展开为二维图像（斜交裂隙呈“正弦状”曲线 [4]），实现高精度可视化数据采集，成为重要探测手段。

但该技术应用仍存挑战：一是图像含岩石纹理、泥浆残留等干扰，与裂隙视觉特征相似，增加识别难度 [5, 6]；二是多数矿山依赖人工判读，单孔分析需 3-5 小时，且结果主观差异大；三是缺乏高效可靠方法实现二维图像到三维地质模型的重构。因此，实现围岩裂隙精准识别与三维重构，兼具理论价值与工程迫切性。本研究拟通过图像处理、数学建模等方法，解决裂隙识别、参数提取、粗糙度评价及多孔连通性分析等关键问题，为煤矿透明地质构建与灾害预警提供支撑 [7, 8, 9]。

1.2 问题重述

基于本文的问题背景与提供的附件数据，本文需要研究解决以下问题：

问题一：基于像素分类的裂隙智能识别

实现对钻孔成像展开图中裂隙像素的自动识别，并区分真实裂隙与岩石纹理、钻进痕

迹、泥浆污染等干扰因素。需要建立数学模型，对图像中的裂隙像素进行分类识别，使用该模型处理附件 1 中的数据，生成二值化识别结果（裂隙为黑色，非裂隙为白色），并重点分析图 1-1、1-2、1-3，并将所有结果以图像附件形式提交。

问题二：“正弦状”裂隙的定量分析建模

对图像中的“正弦状”裂隙进行自动聚类与参数提取，使用正弦函数模型进行表征。建立数学模型，对裂隙进行聚类并拟合公式 (1) 中的参数：振幅 R 、周期 P 、相位 β 、中心线位置 C ，对附件 2 中的数据进行分析，生成相应参数表，重点分析图 2-1、2-2、2-3，并在正文中展示分析过程与结果。

问题三：复杂裂隙的定量分析建模

对非理想“正弦状”的复杂裂隙进行表征，并计算其粗糙度指标（JRC）。建立数学模型，对附件 3 中的复杂裂隙进行表征，提取每条裂隙的轮廓线离散点，计算 JRC 值，讨论不同采样方法对 JRC 计算的影响，分析采样密度、采样方式、裂隙面积等因素，生成参数表，重点分析图 3-1、3-2、3-3。

问题四：多钻孔裂隙网络的连通性分析与三维重构

基于多个钻孔的成像数据，分析裂隙之间的连通性，并进行三维可视化与不确定性评估。建立数学模型，评估相邻钻孔中裂隙的连通概率，结合附件 4 数据，进行连通性分析，并绘制三维结构图，识别当前钻孔布局下的高不确定性区域，并推荐 3 个最优的补充钻孔位置，并说明优先级。

2. 模型假设及关键性符号说明

2.1 模型假设

(1) 假设裂隙在图像中表现为连续的线性低灰度区域，相邻钻孔间裂隙连通性可通过几何关系判断；

(2) 假设裂隙具有明显的方向性和连通性；

(3) 假设干扰因素（泥浆、钻痕等）具有区别于裂隙的特征模式；

(4) 假设裂隙宽度 $> 1\text{mm}$ （对应特定像素数）。

2.2 符号说明

符号	意义
$I(i, j)$	原始图像在位置 (i, j) 处的灰度值
G_x, G_y	x 和 y 方向的梯度分量
f_1, f_2, f_3	灰度特征、梯度特征、纹理特征
M_{final}	最终二值化裂隙分割图
$R P \beta$	正弦裂隙的振幅、周期、相位
C	裂隙中心线位置
$J(\theta)$	残差平方和损失函数
R^2	决定系数，拟合优度指标
Z_2	基于一阶导数的粗糙度参数
Z_3	基于二阶导数的改进粗糙度参数
JRC	裂隙粗糙度系数

3. 问题一的模型建立与求解

3.1 问题一分析

问题一要求我们克服岩石纹理、钻进痕迹及泥浆污染等干扰因素，通过建立数学模型，实现对钻孔成像展开图中裂隙像素的自动识别。本文首先通过已给出的小样本图像进行数据增强。紧接着，对原始图像进行预处理操作，主要包括灰度化、滤波去噪以及对对比度增强，为后续进一步处理提供更高质量的图像。

然后，图像预处理完后分别通过基于特征提取的 K-Means 聚类融合算法、基于方向滤波增强的 Canny 边缘检测以及 YOLOv8 表面裂缝检测三种模型算法分别进行相应裂隙识别。最终，经过比对发现 YOLOv8 表面裂缝检测的算法效果更好，故而，通过基于图像形态学操作的后处理优化决策进一步优化结果图像。最后，凭借二值化裂隙判别决策模型输出问题一的最终结果。

问题一的思路流程图如下图 2 所示：

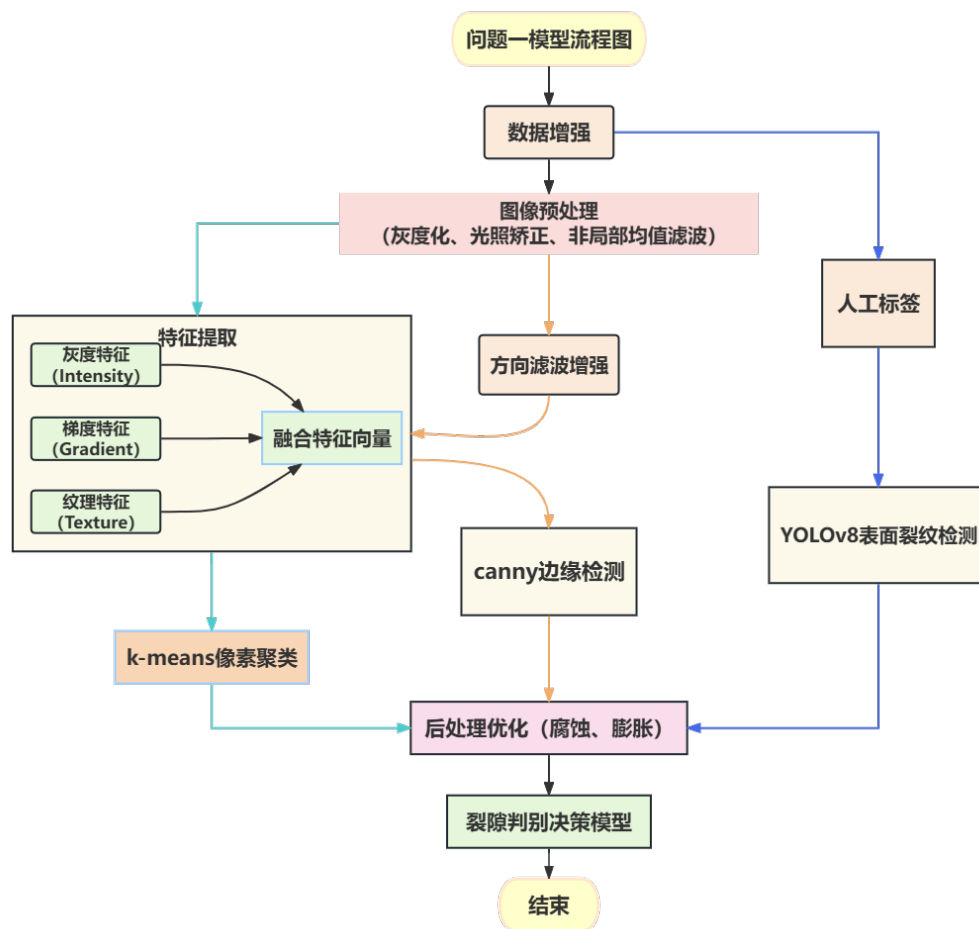


图 2 问题一流程图

3.2 基于小样本的数据增强策略

基于小样本的数据增强算法是一种为解决训练数据稀缺问题而设计的技术。其核心思想是在有限或少量的原始训练样本基础上，人为地创造新的、多样化的训练数据集，进而扩大数据集规模，并且增加数据的多样性，最终避免深度学习模型过拟合，提升其泛化能力和鲁棒性。其算法流程表述如下：

step1：输入少量带标签的训练样本集。

step2：选择增强策略。常见的增强算法通常包括两大类，分别为基于传统图像变换的方法和基于深度学习生成的方法。根据具体任务和数据类型，选择其中一种或上述多种增强技术。

step3：数据生成。使用在线生成或者离线增强生成数据。

step4：模型训练。通过使用原始数据以及增强数据共同进行模型训练。

step5：输出泛化能力以及鲁棒性更强的模型。

3.3 图像预处理

针对钻孔成像展开图中存在岩石纹理、钻进痕迹及泥浆污染等干扰因素，我们先对原始图进行图像预处理，达到抑制噪声，增强对比度，为后续处理提供更高质量的图像。图像预处理流程如下。

1. 灰度化

虽然钻孔成像通常已是灰度图像，但如果输入为彩色图像，需要进行灰度化处理。对于本题钻孔成像，我们使用加权平均法进行灰度化，加权平均法符合人眼视觉特性，绿色通道权重最大，有利于保持细节，并且转换后的灰度图像对比度较好。 (i, j) 为像素坐标，对于图像 $I_{gray}(i, j) = [R(i, j), G(i, j), B(i, j)]$ ，采用**加权平均模型**：

$$I_{gray}(i, j) = 0.299R + 0.587G + 0.114B$$

2. 光照矫正 (Illumination Correction)

由于光源位置、孔壁曲率或镜头污渍导致的亮度分布不均（图像中心亮、四周暗，或出现明显的光斑/阴影）。消除这些问题这一步至关重要，因为不均匀的光照会使得基于阈值的分割算法完全失效——同一裂隙在亮处可能因对比度低而无法识别，在暗处又可能被误认为阴影。

假设图像的真实内容（如裂隙、纹理）是高频信号，而缓慢变化的光照背景是低频信号。因此，可以将图像分解为“光照背景”和“反射细节”两部分。

$$I_{observed}(i, j) = I_{illumination}(i, j) * I_{reflectance}(i, j)$$

我们的目标就是估算并去除 $I_{illumination}$ ，从而得到相对均匀的 $I_{reflectance}$ 。

3. 非局部均值滤波 (NLM)

由于图像在采集、压缩或传输过程中，会产生多种噪声（如高斯噪声、椒盐噪声等），进而导致图像质量下降。滤波去噪作为图像处理中的一项基本且关键的技术，其主要目的是从被噪声干扰的图像中抑制或消除噪声，并且尽可能地保留图像的原始细节和特征，如边缘、纹理等。

本文摒弃传统局部滤波思想，利用图像中存在的全局自相似性进行去噪，采用非局部均值滤波（NLM）去噪处理方法，其基于在整个图像中寻找并对结构相似的区域进行加权平均来估计像素值，从而实现合理去噪与细节保护。其关键步骤与原理如下：

step1：定义窗口。以目标像素为中心划定较大的“搜索窗口”用于寻找相似区域；在搜索窗口内再定义较小的“相似窗口”，作为计算相似度的基本单元；

step2：计算相似度权重。先通过欧氏距离等度量目标块与搜索窗口内各参考块的相似度，常用欧氏距离或均方差（MSE）作为度量标准，公式为：

$$d(B_i, B_j) = \sum_{x,y} (B_i(x, y) - B_j(x, y))^2$$

再用含滤波参数 h （控制权重衰减）的高斯函数将相似度转化为对应权重，通常使用高斯函数：

$$w(x, y) = \exp\left(-\frac{d(B_i, B_j)}{h^2}\right)$$

step3：加权平均更新像素值。对目标像素的估计值，通过所有相似像素块的加权平均计算

$$\hat{I}(x) = \frac{\sum_{y \in \Omega} w(x, y) I(y)}{\sum_{y \in \Omega} w(x, y)}$$

其中 Ω 为搜索窗口， $I(y)$ 为参考像素值。同时确保归一化权重总和为 1，进而避免亮度偏移。

4. 方向滤波增强（Gabor 滤波器组）

在光照矫正、非局部均值滤波之后，需要对特征增强，突出特定方向的裂隙特征，并抑制无关方向的纹理和噪声，我们继续用 Gabor 滤波器组对图像进行方向滤波增强。Gabor 滤波器是一种带通滤波器，其核函数由高斯函数调制正弦平面波构成，能同时在空间域和频率域提供最优的局部化特性。其脉冲响应形状与哺乳动物视觉皮层中简单细胞的 receptive field 非常相似，因此非常适用于纹理分析和边缘检测。在裂隙识别中，我们利用 Gabor 滤波器的方向、频率选择特性，通过使用一组覆盖不同方向的 Gabor 滤波器，我们可以全方位地增强图像中所有可能方向的裂隙。实际应用中，定义二维 Gabor 滤波器的核函数如下：

$$G(x, y) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \cdot \cos\left(2\pi \frac{x'}{\lambda} + \psi\right)$$

定向 Gabor 滤波器并行处理图像，并将结果融合，流程如下：

Algorithm 1 定向 Gabor 滤波增强（保留斜向，抑制正交）

Input: 预处理图像 I , 波长 λ , 相位 $\psi \leftarrow \pi/2$ **Output:** 增强图像 G_{enhanced}

```
1 初始化方向集合  $\Theta \leftarrow \{20^\circ, 30^\circ, 40^\circ, 50^\circ, 60^\circ, 70^\circ, 110^\circ, 120^\circ, 130^\circ, 140^\circ, 150^\circ, 160^\circ\}$ 
2 for  $\theta \in \Theta$  do
3    $G_\theta \leftarrow \text{createGaborKernel}(\lambda, \theta, \psi)$                                 // 创建斜向 Gabor 核
4    $R_\theta \leftarrow |I * G_\theta|$                                                 // 计算绝对值响应
5 end
6  $G_{\text{enhanced}} \leftarrow \max_{\theta \in \Theta}(R_\theta)$                                 // 取斜向最大响应
7 return  $\text{normalize}(G_{\text{enhanced}})$ 
```

5. 图像预处理结果

通过对原始图进行灰度化、光照矫正、非局部均值滤波、方向滤波增强后，得到最终的图使微小裂隙更加区别于其他因素，针对问题一的图像预处理部分结果展示如下图 3 所示：

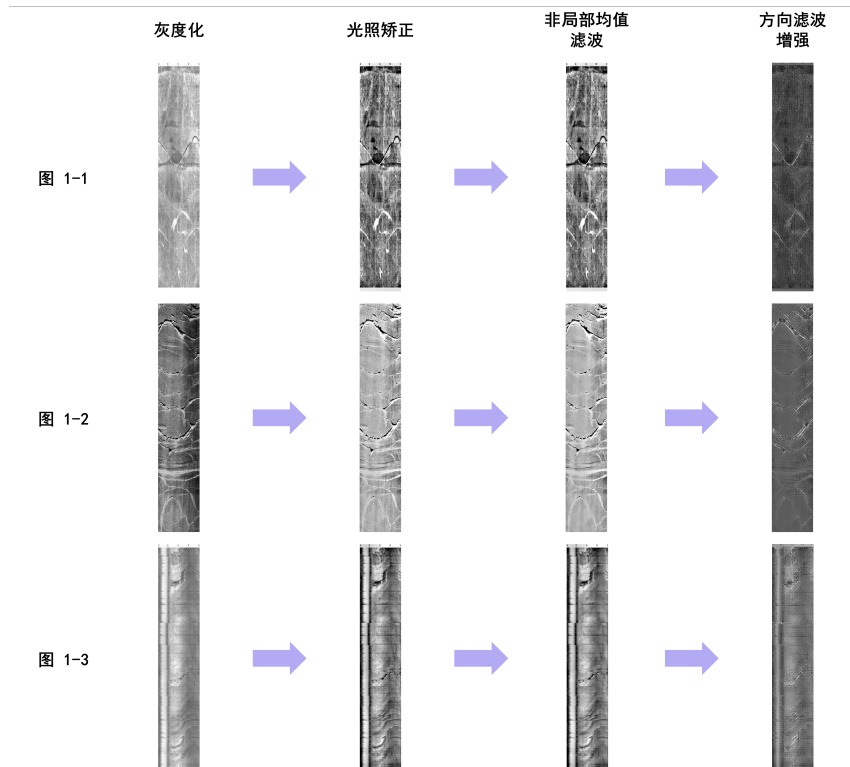


图 3 图像预处理结果对比图

3.4 裂隙判别决策模型

建立二值化裂隙模型 $M_{final} \in (0, 1)^{H \times M}$ 如下：

$$M_{final}(i, j) = \begin{cases} 1, & \text{黑色像素 (裂隙)} \\ 0, & \text{白色像素 (背景)} \end{cases}$$

其中， $M_{final}(i, j) = 1$ 表示该像素被判别为裂隙， $M_{final}(i, j) = 0$ 表示该像素被判别为背景。

3.4.1 多特征融合的 K-Means 聚类融合算法

本算法作为一种综合性较强的图像聚类策略，其核心在于摒弃使用单一图像特征的局限性，转而综合提取并融合图像的灰度、梯度和纹理三类本质特征，构建一个鲁棒性更强、信息量更丰富的联合特征空间。在此空间上执行 K-Means 聚类，通过克服光照变化、噪声干扰和表面材质差异等挑战，进而实现更为精确、更符合视觉感知的图像区域划分。

1. 特征提取

灰度特征是最直观、最简单的特征。裂隙由于是孔壁上的物理划痕或裂缝，其表面通常更粗糙或充满填充物（如空气、水、泥浆），对光的反射率低于周围完整的岩壁。因此，在图像中，裂隙像素的灰度值通常显著低于其周围岩石基质的像素。输入经过预处理（灰度化、光照矫正、非局部均值滤波）后的灰度图像 I ，对于图像中的每一个像素点 (i, j) ，其灰度特征 $f_1(i, j)$ 就是它自身的灰度值，即

$$f_1(i, j) = I(i, j)$$

进而输出与原始图像同尺寸的特征图，其中每个位置的值就是该点的灰度值。单独使用灰度特征极易受干扰。例如，黑色的泥浆痕迹、深色的矿物纹理或光照产生的阴影也都是暗色的，会产生大量假正例 (False Positives)。因此，必须结合其他特征来排除这些干扰。

梯度特征 (Gradient Feature) 是用于衡量裂隙与岩石的交界处，发生剧烈变化强度和方向的指标。裂隙的边缘通常表现为两侧像素灰度值差异很大。预处理（灰度化、光照矫正、非局部均值滤波）后的灰度图像 I 。使用 Sobel 算子与图像进行卷积，计算每个像素点在 x 方向（水平）和 y 方向（垂直）上的梯度近似值。Sobel 算子卷积核：

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

卷积计算（* 表示卷积操作）：

$$g_x(i, j) = G_x * I(i, j), \quad g_y(i, j) = G_y * I(i, j)$$

梯度幅值 (Edge Strength):

$$f_2(i, j) = \sqrt{g_x(i, j)^2 + g_y(i, j)^2}$$

梯度方向 (Edge Orientation):

$$\theta(i, j) = \arctan 2(g_y(i, j), g_x(i, j))$$

输出梯度幅值图作为特征。该图像中，边缘处的像素值很高（亮），而平坦区域的像素值很低（暗）。钻进痕迹、刀具刮擦等线性干扰也会产生强烈的梯度响应，造成混淆。此时需要结合纹理特征或后续的形态学处理来区分。

纹理特征 (Texture Feature)，岩石背景通常具有某种特定、重复的颗粒状或层理状纹理模式，而裂隙则会打破这种纹理的连续性。纹理特征旨在量化每个像素周围区域的这种模式规律性，从而将均匀的岩石背景与异常的裂隙区分开来。同上的输入相同预处理后的灰度图像，对于一个以像素 (i_c, j_c) 为中心的 3×3 邻域，将中心像素的灰度值 g_c 与周围 8 个邻域像素的灰度值 $g_p (p = 0, 1, \dots, 7)$ 进行比较。根据比较结果生成一个 8 位的二进制数：

$$s(g_p, g_c) = \begin{cases} 1 & \text{if } g_p \geq g_c \\ 0 & \text{if } g_p < g_c \end{cases}$$

将这个二进制数转换为十进制数，即为该中心像素的 LBP 值：

$$f_3(i_c, j_c) = \sum_{p=0}^7 s(g_p, g_c) \cdot 2^p$$

这个 LBP 值（通常在 0 - 255 之间）编码了该点周围的局部纹理模式。输出 LBP 特征图，其中每个像素的值代表了其周围的纹理模式。纹理特征是区分裂隙与岩石纹理的关键。裂隙区域的 LBP 模式通常与周围大面积的岩石背景模式显著不同。它对于光照变化不敏感，非常稳健。

特征融合与最终决策，对于本文问题，我们需要将以上三个特征进行融合 (Fusion)，形成一个综合的特征向量，从而做出更可靠的决策。对于每个像素 (i, j) ，我们可以构造一个三维特征向量：

$$f(i, j) = [f_1(i, j), f_2(i, j), f_3(i, j)]^T$$

特征提取后的可视化（以附件 1 中图4为例）如下：

一个像素被判定为裂隙，可能需要同时满足：灰度很低、梯度幅值较高、且纹理模式与周围背景差异大。通过这种多特征融合的策略，系统能够综合利用裂隙的颜色、形状和结构信息，极大地提高了在复杂干扰背景下准确识别裂隙的能力。

2.K-Means 聚类

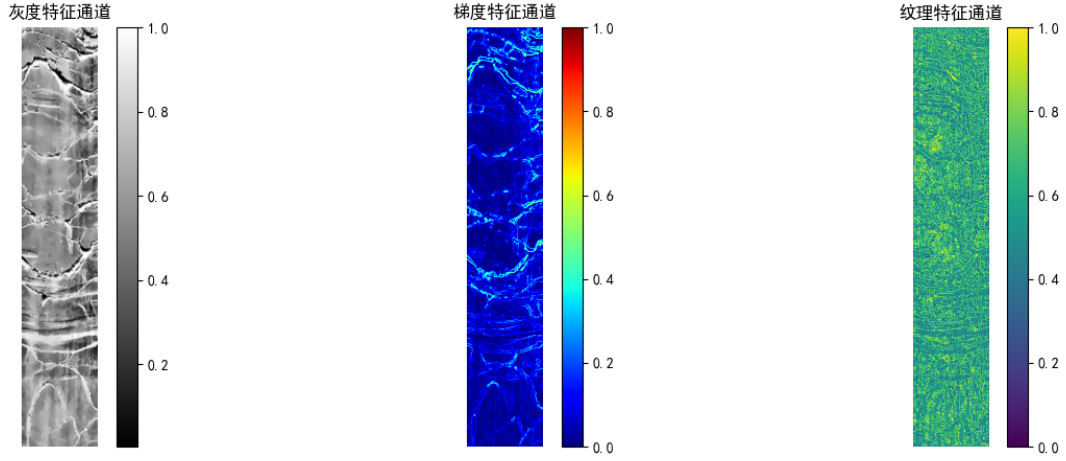


图 4 三个特征融合后的钻孔成像展开图

K-means 是一种经典的无监督聚类算法，其目标是将数据点划分到 K 个簇中，使得同一簇内的点彼此相似，而不同簇的点相异。在图像分割中，每个“数据点”就是一个像素的特征向量。

Step1: 构建像素特征向量。输入预处理和特征提取后的图像。假设图像尺寸为 $H \times W$ 。图像中的每一个像素都是一个样本点，总样本数为 $N = H \times W$ 。特征向量 (Feature Vector) 用一个向量 f_i 表示，这个向量可以包含灰度强度 (Intensity)($f_1 = I(x, y)$)、梯度幅值 (Gradient Magnitude)($f_2 = G(x, y)$)、纹理特征 (Textures)(f_3)。

Step2: 进行初始化，确定簇的数量 (K)，对于裂隙分割 $K=5$ 。随机初始化质心 (Centroids)，随机选择 K 个像素点的特征向量作为初始的聚类中心 $\mu_1, \mu_2, \dots, \mu_K$ 。

Step3: 进行迭代优化 K-means 核心循环，算法会重复执行分配、更新两个步骤，直到质心的位置不再发生显著变化为止。分配对于每一个像素点 i ，计算其特征向量 f_i 到所有 K 个质心 μ_j 的距离。最常用的距离度量是欧氏距离 (Euclidean Distance)。 $d_{ij} = \|f_i - \mu_j\|^2$ ，将像素点 i 分配到距离它最近的那个质心所代表的簇中。 $c_i = \arg \min_j \|f_i - \mu_j\|^2$ (c_i 表示像素 i 所属的簇标签)。更新对于每一个簇 j ，重新计算该簇新的质心 μ_j^{new} 。新的质心是该簇内所有像素点特征向量的平均值。 $\mu_j^{new} = \frac{1}{|C_j|} \sum_{i \in C_j} f_i$ ($|C_j|$ 是属于簇 j 的像素个数)。当一次迭代中，所有质心的移动距离小于某个阈值，或者达到最大迭代次数时，算法停止。输出每个像素点都有一个最终的簇标签 c_i $1, 2, \dots, K$ 。

Step4: 识别裂隙，裂隙区域具有较低的灰度值 (暗) 和较高的梯度值 (边缘明显)。我们可以查看各个簇的最终质心向量。假设特征向量是 [灰度, 梯度]，那么那个质心为 [低值, 高值] 的簇就是裂隙簇。

3.4.2 多特征融合的 Canny 边缘检测

Canny 边缘检测是一个多阶段的经典算法，其目标是找到图像中真正的边缘，并尽可能抑制噪声。它将直接输出一个二值图，其中白色像素代表边缘 (即我们候选的裂隙)。针

对问题一，我们将建立一个基于 Canny 边缘检测的裂隙判别决策模型。这个模型的输入是基于图像预处理后的图像，输出最终的二值化裂隙分割图。

对于裂隙判别决策，我们输入预处理后的灰度图像 $I_{preprocessed} \in \mathbf{R}^{H \times M}$ ，其中每个像素 $I(i, j) \in [0, 255]$ 寻找一个映射 $F_{decision}$ ，使得 $M_{final} = F_{decision}(I_{preprocessed})$ 尽可能地准确识别出真实裂隙，并抑制所有干扰。通过一系列图像处理操作符将 Canny 边缘检测结果逐步优化为最终的裂隙二值图，利用 canny 边缘检测找到所有可能的边缘候选点，其决策规则基于梯度幅值，**初级边缘检测决策 (Canny Edge Detection)** 步骤如下：

Step1: 梯度计算与决策。计算图像 I 在 x 和 y 方向的梯度 $G_x = K_x * I$, $G_y = K_y * I$ ，其中 K_x, K_y 是 Sobel 算子。计算梯度幅值 $G = \sqrt{G_x^2 + G_y^2}$ 及梯度方向 $\theta = \arctan 2(G_y, G_x)$ (并量化到 $0^\circ, 45^\circ, 90^\circ, 135^\circ$ 四个方向)。

Step2: 非极大值抑制 (NMS)。对于每个像素点，比较其梯度方向 (i, j) 上的两个邻接点的梯度幅值 G 。局部极大值决策表示为：

$$G_{sup}(i, j) = \begin{cases} G(i, j) & \text{if } G(i, j) \geq G_{neighbor1} \text{ and } G(i, j) \geq G_{neighbor2} \\ 0 & \text{otherwise} \end{cases}$$

确保只保留梯度方向上局部最强的响应，细化边缘。

Step3: 双阈值滞后决策 (Hysteresis Thresholding)。首先设定两个阈值：高阈值 T_{high} ，低阈值 T_{low} ($T_{high} > T_{low}$)。设置初步决策：

$$E_{strong}(i, j) = \begin{cases} 1, & \text{if } G_{sup}(i, j) \geq T_{high} \\ 0, & \text{otherwise} \end{cases}$$

$$E_{weak}(i, j) = \begin{cases} 1, & \text{if } T_{low} \leq G_{sup}(i, j) < T_{high} \\ 0, & \text{otherwise} \end{cases}$$

边缘连接决策从所有 $E_{strong} = 1$ 的像素点出发，递归地将其 8 邻域内所有 $E_{weak} = 1$ 的像素点重新决策为强边缘 (赋值为 1)。这是一个基于连通性的决策过程。最终 Canny 输出：

$$E_{canny}(i, j) = \begin{cases} 1, & \text{if } (i, j) \in E_{strong} \text{ or } (i, j) \in \text{connected weak edges} \\ 0, & \text{otherwise} \end{cases}$$

我们得到了初级的二值边缘决策图 E_{canny} 。

3.4.3 YOLOv8 表面裂隙检测

YOLOv8 表面裂隙检测算法是一种基于深度学习，从端到端的目标检测解决方案，它采用了最新的 YOLOv8 架构，专门用于在复杂背景下的各种材料表面（如金属、陶瓷等）

上快速、准确地定位和识别裂纹缺陷。设输入图像为 $I \in \mathbb{R}^{H \times W \times 3}$ ，构建 YOLOv8 模型可表示为从图像空间到检测结果的映射函数如下：

$$f_{\text{YOLOv8}} : I \rightarrow \mathcal{D}$$

其中检测结果 $\mathcal{D} = d_{i=1}^N$ ，每个检测结果 $d_i = (b_i, c_i, s_i)$ 包含边界框 b_i 、类别置信度 c_i 和检测得分 s_i 。

特征提取过程按照上面过程进行，得到三维融合特征向量 $f(i, j)$ ，对于每个特征层级 l ，检测头产生预测张量：

$$P_l = f_{\text{head}}(f_l) \in \mathbb{R}^{H_l \times W_l \times (5 + N_{\text{classes}})}$$

其中每个网格位置 (i, j) 预测 N_{anchors} 个检测结果，每个预测包含边界框偏移量 $(\Delta x, \Delta y, \Delta w, \Delta h)$ 、目标置信度 o 、类别概率分布 $\mathbf{p} = (p_1, p_2, \dots, p_{N_{\text{classes}}})$ 。

构建 **CIoU (Complete IoU) 损失函数**：

$$\mathcal{L}_{\text{box}} = 1 - \text{IoU} + \frac{\rho^2(b_{\text{pred}}, b_{\text{gt}})}{c^2} + \alpha v$$

其中， ρ 为欧氏距离， c 为最小外接矩形对角线长度 v 为宽高比一致性度量 α 为权衡参数。

3.5 后处理优化决策 (Post-Processing)

图像形态学操作是一种基于形状的图像处理技巧。其基本思想是通过结构元素在图像上系统性地移动，同时检查结构元素与图像重叠区域的像素关系，来获取图像的形状信息，进而达到净化图像、提取特征、分割目标的目的。基于裂隙的几何特征，对初级决策进行优化，剔除误判，连接漏判。

1. 形态学闭操作决策。连接由于噪声或光照不均而断裂的裂隙段。使用一个结构元素 B （例如水平方向的线形元素，大小为 1×5 ）对图像进行闭运算（先膨胀后腐蚀）。

$$E_{\text{closed}} = (E_{\text{canny}} \oplus B) \ominus B$$

断裂间距小于结构元素长度的裂隙会被连接起来。

2. 面积滤波决策。根据连通区域的面积剔除小噪声，先进行连通组件标记，找到所有连通区域 $R_k, k = 1, 2, \dots, K$ ，计算每个区域的像素面积 $A(R_k)$ ，设定一个面积阈值 T_{area} ，建立面积决策为：

$$E_{\text{area}}(i, j) = \begin{cases} 0, & \text{if } (i, j) \in R_k \text{ and } A(R_k) < T_{\text{area}} \\ E_{\text{closed}}(i, j), & \text{otherwise} \end{cases}$$

将面积小的孤立噪声区域决策为背景（0）。

3. 骨架化决策 (Skeletonization)。将可能因膨胀而变粗的裂隙细化成单像素宽的中心线, 更符合“裂隙”的物理形态。使用迭代算法 (如 Zhang-Suen 算法) 不断剥离边缘像素, 直到所有区域都变为单像素宽, 且不改变其拓扑结构。输出 $E_{skeleton} = Skeletonize(E_{area})$ 表示最终的决策结果 M_{final} 。

3.6 方案对比与结果优化

3.6.1 方案对比

面对附件 1 中图 1-1、图 1-2 及图 1-3, 分别通过上述三种模型方法, 即基于的多特征融合的 Canny 边缘检测与 K-Means 聚类算法, 以及 YOLOv8 裂隙检测进行裂隙识别, 其结果如图5 所示:

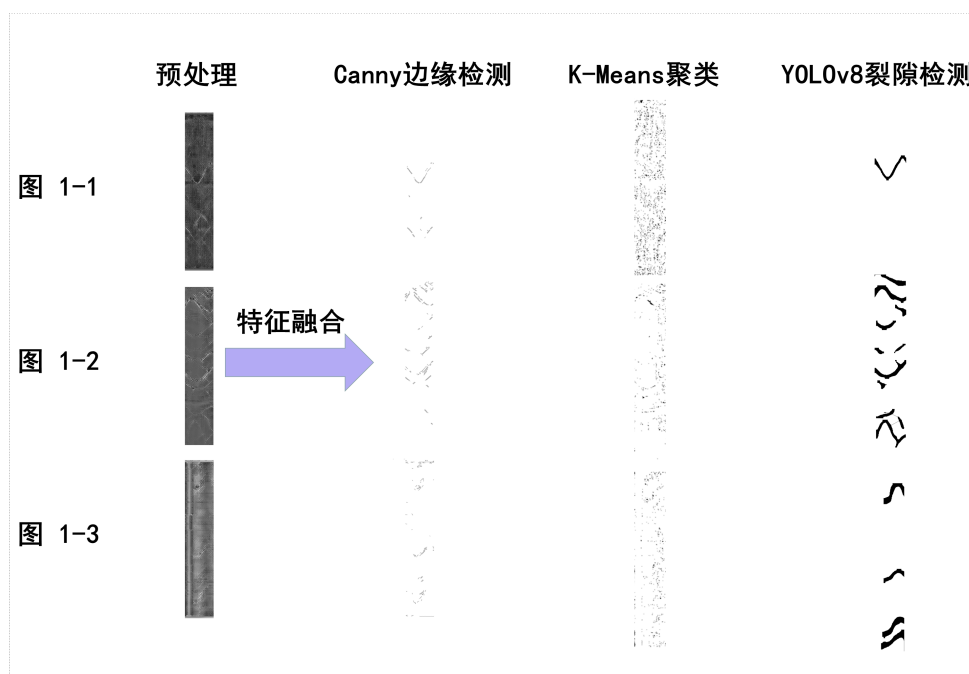


图 5 问题一结果对比图

图中呈现了裂隙识别流程及成果。图像以三行分别对应附件 1 里的图 1-1、图 1-2、图 1-3 三幅原始钻孔图像的处理过程, 每行包含预处理后图像、经特征融合后的 Canny 边缘检测结果及 K-Means 聚类结果、YOLOv8 裂隙检测结果四个子图 (从左至右)。图像中白色像素对应背景岩石基质, 黑色像素对应裂隙区域。

3.6.2 结果优化

canny 边缘决策及后处理优化的裂隙判别模型包含了基于梯度幅值、非极大值抑制和双阈值的多层决策, canny 的双阈值会滤除大部分岩石纹理, 钻进痕迹是线性的, 会产生强梯度响应。但它们通常方向单一 (如沿钻进方向)。通过选择非水平的结构元素进行闭运算, 可以避免连接这些垂直痕迹, 从而在后续面积滤波中将其作为孤立短线剔除。有效对

抗泥浆污染干扰。同时结合后处理优化策略，最终以二值化图像形式呈现，其结果如图6所示：

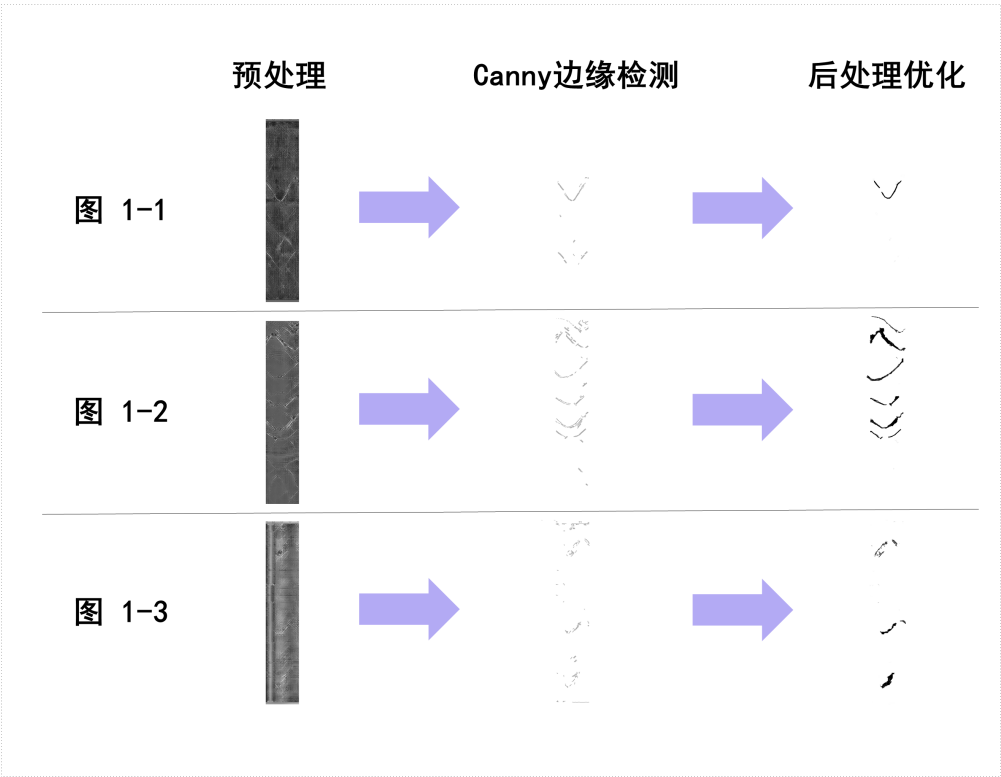


图 6 后处理优化后的 Canny 边缘检测结果呈现

上图表明进行后处理优化后识别裂隙效果确实增强。最终对附件 1 中图 1-1、1-2、1-3 识别的裂隙数量分别为 1、6、3。

3.7 问题小结

问题一聚焦钻孔成像中的裂隙识别问题，系统实现了三类技术方案：基于传统图像处理的 K-means 聚类、Canny 边缘检测，以及基于深度学习的 YOLOv8 目标检测。通过多维度结果对比可知，相较于传统方法的性能，YOLOv8 裂隙判别模型在该任务中展现出一定优势。然而，结合钻孔成像裂隙识别任务的实际落地需求，本研究最终选择 Canny 边缘检测等传统方法。核心原因在于 YOLOv8 的性能发挥极度依赖高质量人工标注数据，而钻孔成像数据往往具有样本量大、裂隙形态多样、标注标准难统一等挑战，大规模精准标注不仅会产生高昂的时间与人力成本，还可能因标注主观性引入误差。相较而言，Canny 边缘检测更且具备清晰的算法逻辑与强可解释性，在实际应用中易于调试优化，能够更好地满足特定场景下的裂隙识别任务要求。

4. 问题二的模型建立与求解

4.1 问题二分析

问题二要求我们在问题一裂隙识别的基础上，克服裂隙形态复杂性和测量噪声干扰，通过建立数学模型，实现对”正弦状”裂隙的自动聚类与参数化表征。本文首先基于问题一方法输出的二值化裂隙图像，对附件 2 图像进行预处理，用多特征融合进行 canny 边缘检测，得到同样的二值边缘图像。在这之后，需要将同一条裂隙边缘像素进行聚类，采用方向一致及空间接近性判断聚类准则，提取各裂隙实例的中心线坐标数据。构建“正弦”参数反演模型，通过最小化残差平方和拟合模型参数（振幅 R 、周期 P 、相位 β 以及中心线位置 C ）。

然后，对预处理后的裂隙数据分别通过基于梯度下降-牛顿法混合优化算法和神经网络参数回归模型两种方法进行正弦参数反演。最终，经过对比评估发现神经网络参数回归模型在大多数情况下拟合效果更优，故而，通过决定系数 R^2 进行拟合优度检验，确保参数估计的可靠性。最后，根据最优参数生成裂隙的完整数学表征，并分析其地质意义。

问题二的思路流程图如下图 7 所示：

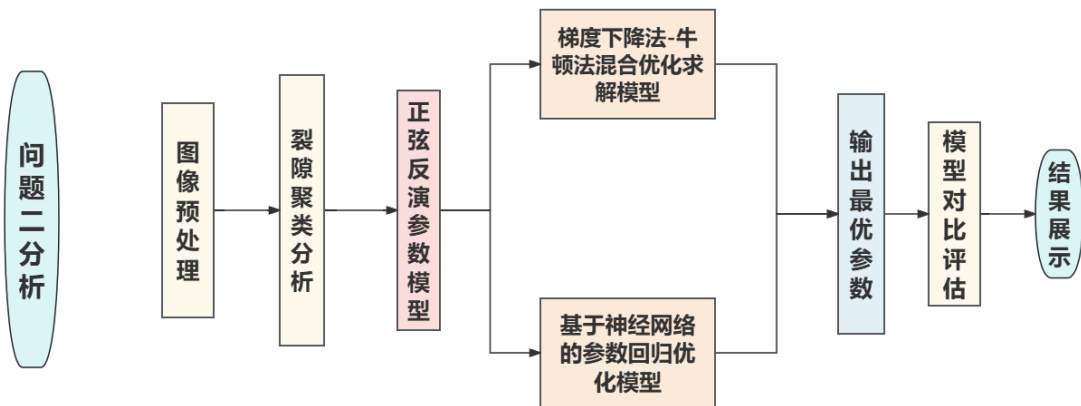


图 7 问题二流程图

4.2 模型准备

4.2.1 图像准备

在问题 1 中，我们通过 canny 边缘检测算法与图像形态学后处理相结合的方式，成功实现了对钻孔成像展开图中裂隙像素的自动识别。该模型的最终输出结果为一组二值化图像，其中被识别为裂隙的像素点被标记为黑色（像素值为 0），而背景及其他非裂隙区域则被标记为白色（像素值为 255）。这些二值图像构成了问题 2 定量分析的输入数据基础。对于问题 2 的钻孔成像图像，我们依据问题 1 的裂隙自动识别模型，将附件 2 同样形成一

系列二值化图像（裂隙像素为黑色，背景为白色）。经过 Canny 检测后，得到二值边缘图像 $E(x, y)$ ，其中边缘像素值为 1，非边缘像素值为 0。

4.2.2 裂隙聚类分析

边缘检测结果包含各类线性特征，需要将属于同一条裂隙的边缘像素进行聚类。canny 边缘检测的每个边缘像素梯度同问题 1，我们采用基于**方向一致性**和**空间接近性**的聚类算法，定义两个聚类判断准则分别为：

1. 空间接近性

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} < d_{\max}$$

其中 $d_{\max} = 10$ 像素（约 1.5mm），确保聚类像素空间连续。

2. 方向一致性

$$\Delta\theta_{ij} = \min(|\theta_i - \theta_j|, 2\pi - |\theta_i - \theta_j|) < \theta_{\max}$$

其中 $\theta_{\max} = 30^\circ$ ，确保同一裂隙像素方向相近。

具体算法以边缘图像 E 和方向图 θ 为输入，输出裂隙标签图 L 。首先，初始化标签图 L 为零矩阵，设置当前标签 $\text{label} = 1$ 。接着，遍历所有边缘像素 (x_i, y_i) ，若该像素在标签图 L 中未被标记（值为 0），则创建新聚类 C_{label} 并将该像素加入其中，同时初始化队列 Q 并把该像素放入队列。然后，当队列 Q 不为空时，取出队列中的像素 p ，在其邻域内寻找满足空间距离 $d(p, q) < d_{\max}$ 且方向差 $\Delta\theta(p, q) < \theta_{\max}$ 的未标记边缘像素 q ，将找到的 q 加入聚类 C_{label} ，标记 $L(q) = \text{label}$ 并放入队列 Q ，之后将当前标签 label 加 1。最后，过滤掉像素数少于 $N_{\min} = 20$ 的小聚类（以去除噪声），返回最终的标签图 L 。

对于每个聚类 C_k （代表一条裂隙），提取其中心线点集 S_k 。对于聚类 C_k ，计算协方差矩阵，求特征向量确定主方向沿主方向将裂隙均匀分为 M 段（ $M = \lceil \text{长度}/5 \rceil$ ，每段约 5 像素），对第 m 段提取该段内所有像素 $(x_i, y_i)_{i=1}^{N_m}$ ，计算质心 $(\bar{x}_m, \bar{y}_m) = \left(\frac{1}{N_m} \sum x_i, \frac{1}{N_m} \sum y_i \right)$ ，得到中心线点集 $S_k = (\bar{x}_m, \bar{y}_m)_{m=1}^M$ 。

在经过对附件 2 中图像特征提取、canny 边缘检测、后处理优化、GMM 聚类分析以及裂隙聚类完后，得到最终的裂隙提取图如下图 8（以附件 2 中图 2-1、2-2、2-3 为例）：

4.2.3 GMM 聚类算法

高斯混合模型（Gaussian Mixture Model, GMM）作为一种概率生成模型，其核心思想是：假设其中的数据集由多个不同参数的高斯分布（正态分布）混合生成，凭借拟合这些高斯分布下的参数，进而实现对数据分布的建模、聚类或概率密度估计。GMM 的本质是用多个单峰的高斯分布“叠加”出复杂的多峰数据分布。具体包含两个核心组件：

1. 模型数学表达式：对于一个 D 维数据样本 x ，其概率密度函数（PDF）由 K 个高斯分布加权求和得到：

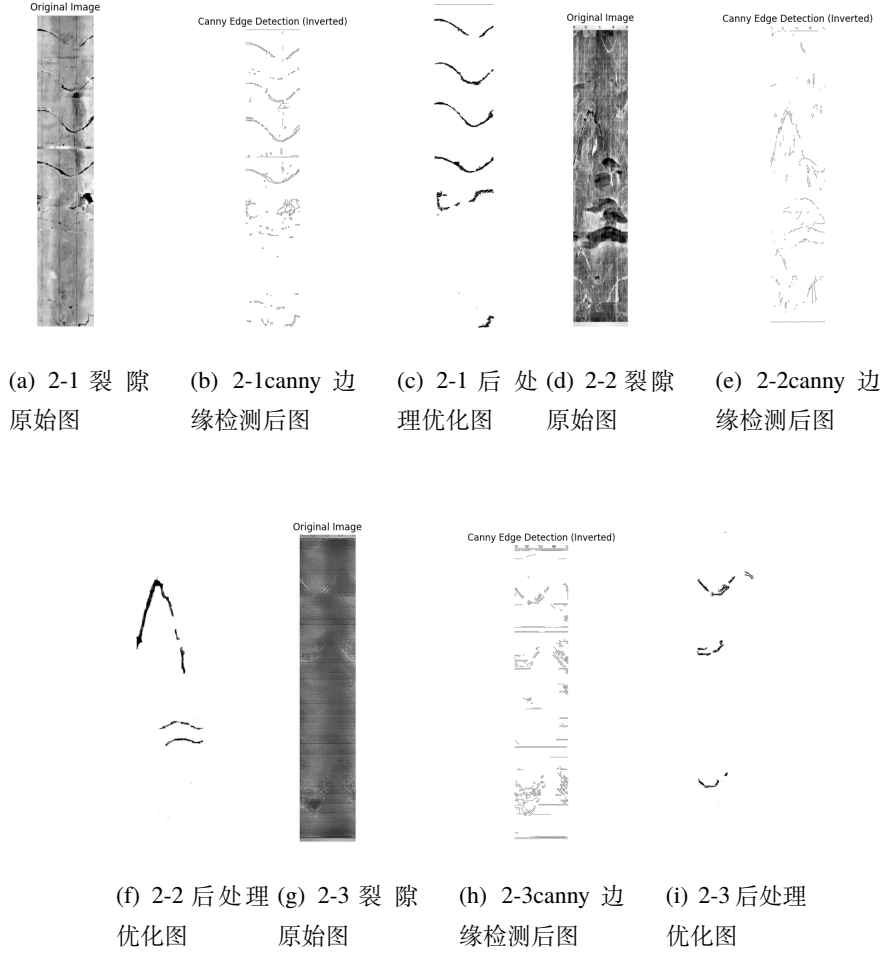


图 8 2-1 最终处理后结果图

$$p(x|\theta) = \sum_{k=1}^K \alpha_k \cdot \mathcal{N}(x|\mu_k, \Sigma_k)$$

其中, K 表示为之预设的高斯分布数量 (即对应聚类任务中的簇数); α_k 表示第 k 个高斯分布的“混合权重”, 满足 $0 \leq \alpha_k \leq 1$ 且 $\sum_{k=1}^K \alpha_k = 1$ (权重反映的是该分量对整体分布的贡献); $\mathcal{N}(x|\mu_k, \Sigma_k)$ 的含义是第 k 个 D 维高斯分布, 其中 $\mu_k \in \mathbb{R}^D$ 是第 k 个分量的均值向量 (决定分布中心), $\Sigma_k \in \mathbb{R}^{D \times D}$ 是第 k 个分量的协方差矩阵 (决定分布的形状、大小和旋转方向), $|\Sigma_k|$ 是协方差矩阵的行列式。 $\theta = \{\alpha_1, \dots, \alpha_K; \mu_1, \dots, \mu_K; \Sigma_1, \dots, \Sigma_K\}$ 表示 GMM 的所有待估参数。

2. 潜在变量解释: GMM 的“生成逻辑”可通过引入潜在变量 z , 代表样本来自第 k 个高斯分量, 从权重分布 $\text{Multinomial}(\alpha_1, \dots, \alpha_K)$ 中随机选择一个分量 k (概率为 α_k); 从第 k 个高斯分布 $\mathcal{N}(x|\mu_k, \Sigma_k)$ 中随机生成样本 x ; 潜在变量 z 的后验概率 $p(z = k|x, \theta)$ 表示“样本 x 来自第 k 个分量的概率”) 被称为责任 (Responsibility), 记为 γ_{nk} , 对第 n 个样本, $\gamma_{nk} = p(z_n = k|x_n, \theta) = \frac{\alpha_k \cdot \mathcal{N}(x_n|\mu_k, \Sigma_k)}{\sum_{j=1}^K \alpha_j \cdot \mathcal{N}(x_n|\mu_j, \Sigma_j)}$, 这一概率是 GMM 聚类的核心依据——

若 γ_{nk} 最大，则将样本 x_n 归为第 k 类。

通过拟合 GMM 算法模型，我们能够得到每个像素点属于每个类别的概率，并将其分配给概率最大的那个类别，同时用该类别的代表色进行渲染，结合模型其输出图9如下（以附件 2 中图 2-1、2-2、2-3 为例）：

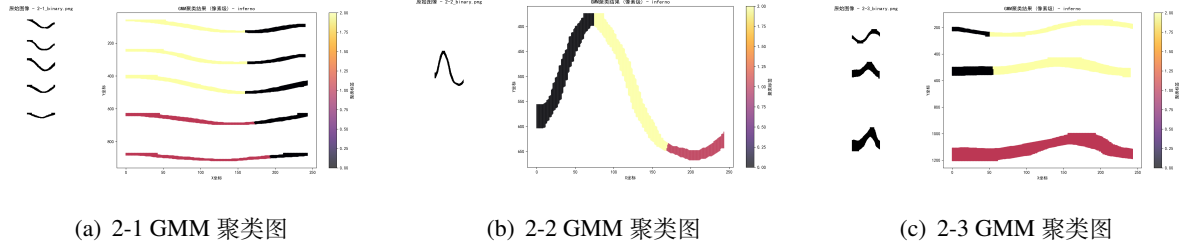


图 9 聚类算法处理结果图

上图展示出附件 2 中裂隙个数分别为 5（图 2-1）、1（图 2-1）、3（图 2-1）。

4.3 建立“正弦”参数反演模型

在钻孔成像展开图的二维平面中，建立坐标系 X-Y，横轴 x 表示钻孔周向展开距离（单位：mm，范围 0 钻孔周长），纵轴 y 表示钻孔轴向深度（单位：mm），对于分离出的单条裂隙，我们得到一组描述其中线的离散点集 $S = (x_i, y_i) | i = 1, 2, \dots, N$ ，定义待优化的参数向量 $\theta = [R, P, \beta, C]^T$ 。基于此坐标系，将“正弦状”裂隙用以下“正弦”进行表征：

$$f(x_i; \theta) = R \cdot \sin\left(\frac{2\pi x_i}{P} + \beta\right) + C$$

其中， R 为振幅 $R > 0$ （单位：mm）， P 为周期 $P > 0$ （对于完整的正弦状裂隙，等同于钻孔周长，单位：mm）， β 为相位（单位：rad）， C 为中心线位置（单位：mm）。

我们的目标是找到一组参数 θ^* ，使得预测值与真实值的残差平方和（Sum of Squared Residuals, SSR）最小。因此，目标函数 $J(\theta)$ 定义为：

$$J(\theta) = \sum_{i=1}^N [f_i - f(x_i; \theta)]^2$$

4.4 模型求解

4.4.1 基于梯度下降-牛顿法混合优化求解模型

最小二乘法（又称最小平方法）作为一种数学优化方法。其核心目标是最小化误差平方和（Sum of Squared Errors, SSE）求解模型参数。直接求解 $J(\theta)$ 的解析解（即令梯度为 0 的方程）非常困难，因为模型关于参数是非线性的。我们采用数值优化方法。单一的梯度下降法在远离最优点时收敛慢，而牛顿法在接近最优点时收敛快但需要计算海森矩

阵 (Hessian Matrix)，结合两者可以取长补短。第一阶段用梯度下降法全局搜索，第二阶段用牛顿法局部精细搜索。

梯度下降法进行初步优化是从一个初始猜测 $\theta^{(0)}$ 开始，快速下降至最优解附近区域。梯度下降法迭代公式为：

$$\theta^{(k+1)} = \theta^{(k)} - \eta \cdot \nabla J(\theta^{(k)})$$

其中， k 是迭代次数， η 是学习率（步长）， $\nabla J(\theta)$ 是目标函数的梯度。

当梯度下降的迭代改善（如 $|J(\theta^{(k+1)}) - J(\theta^{(k)})|$ ）小于一个阈值 ϵ_1 ，或达到最大迭代次数 K_{GD} 时，切换至牛顿法。**牛顿法**利用目标函数的二阶导数（海森矩阵 $H(\theta)$ ）信息，能更快地收敛到局部最优点。其迭代公式为：

$$\theta^{(k+1)} = \theta^{(k)} - [H(\theta^{(k)})]^{-1} \cdot \nabla J(\theta^{(k)})$$

当 $\|\nabla J(\theta^{(k)})\|_2 < \epsilon_1$ 或 $k > K_{GD}$ 时，从梯度下降切换至牛顿法。当 $\|\nabla J(\theta^{(k)})\|_2 < \epsilon_2$ （梯度足够小）或 $|J(\theta^{(k+1)}) - J(\theta^{(k)})| < \epsilon_3$ （损失变化足够小）或达到总最大迭代次数 K_{max} 时，算法终止。算法收敛后，输出最优参数 $\theta^* = [R^*, P^*, \beta^*, C^*]^T$ 。算法流程图如图10：

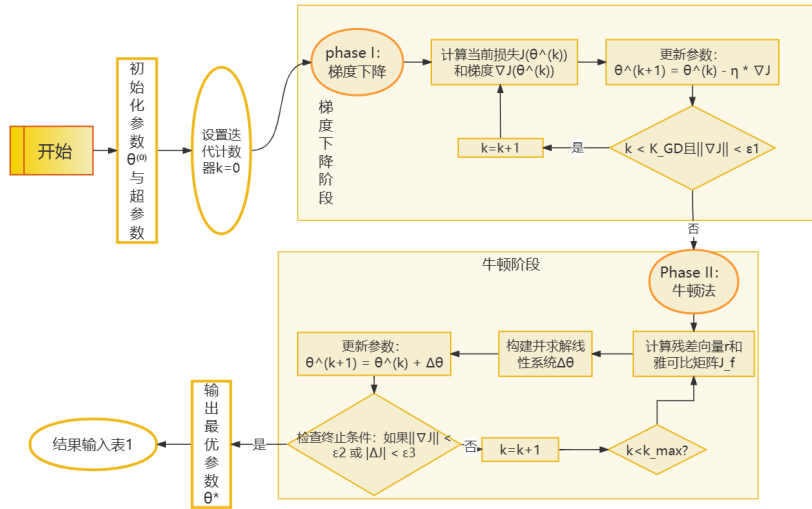
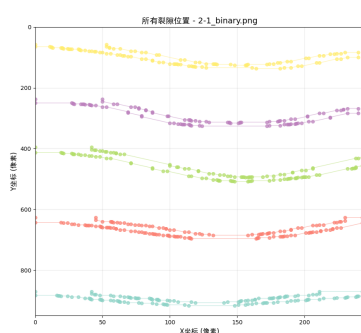


图 10 梯度下降法-牛顿法算法流程图

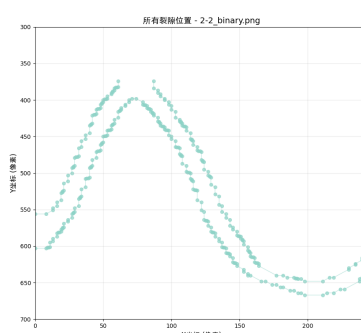
针对附件 2 中的裂隙图像，我们采用了梯度下降法与牛顿法混合优化策略，对”正弦状”裂隙进行最小二乘拟合。对附件 2-1(图 a) 钻孔成像展开图进行裂隙自动识别和正弦曲线拟合，共识别出 5 条主要裂隙，并成功完成了参数化表征。下图11展示了拟合结果与原始图像的叠加显示：

不同裂隙拟合效果不同，以上图中 (a) 为例，对其关键裂隙进行详细分析，下图12展示了裂隙 3、4 的具体拟合细节如下图12：

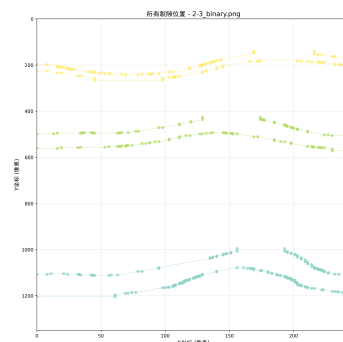
上图中裂隙 3、4 得到的参数描述了一条质量很高、形态规则的裂隙，我们用 R^2 评估回归模型拟合优度的核心指标，其中 R^2 高证明了正弦模型对该裂隙的优秀拟合能力。这



(a) 图 2-1 裂隙拟合标签

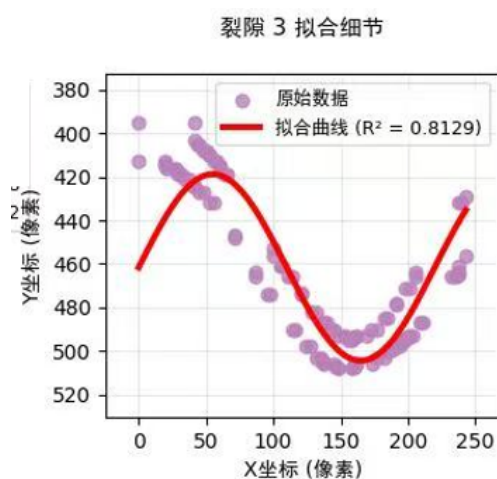


(b) 图 2-2 裂隙拟合标签

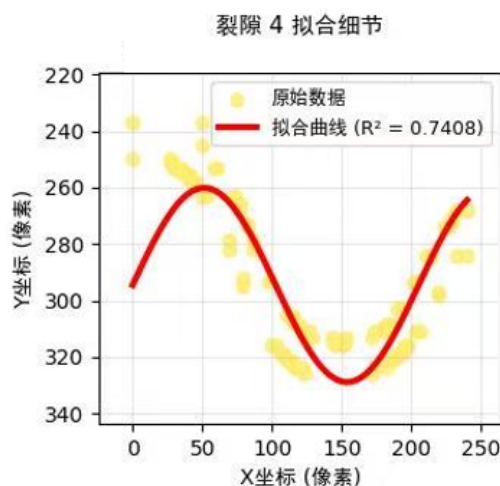


(c) 图 2-3 裂隙拟合标签

图 11 裂隙拟合结果图



(a) 裂隙 3 拟合细节



(b) 裂隙 4 拟合细节

图 12 附件 2-1 裂隙细节拟合结果图

一结果为后续的三维重构和稳定性分析提供了可靠的参数基础，在工程实践中具有重要的指导意义。

为了进一步描述拟合的整体效果，本节给出了梯度下降及拟牛顿法的残差拟合残差图，如下图13所示：

该图呈现了裂隙 1 至裂隙 5（分别以五种颜色区分）的残差随拟合值的分布态势。从整体层面分析，不同裂隙的残差分布存在明显差异：裂隙 1 的残差大多聚集在 0 线附近，离散程度较小，表明拟合效果较为理想；裂隙 2、裂隙 3 的残差离散程度也相对可控，拟合结果与实际值的契合度较高；然而，裂隙 4、裂隙 5 的残差离散程度较大，且存在残差绝对值较大的点，这反映出对这两条裂隙的拟合效果稍显逊色，推测可能与裂隙自身的形态、纹理等特征因素存在关联。

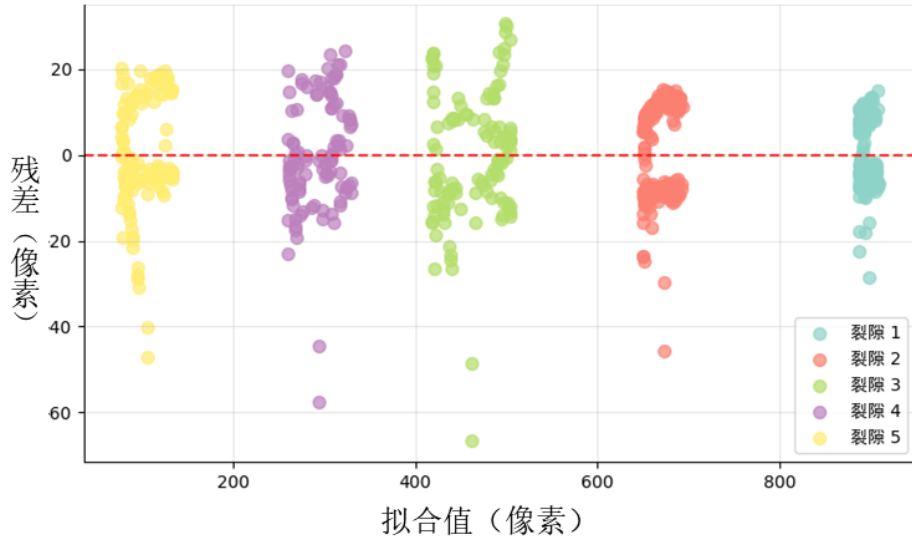


图 13 基于梯度下降 + 拟牛顿法的拟合残差图

4.4.2 基于神经网络的参数回归优化模型

针对“正弦状”裂隙的四个参数（振幅 R 、周期 P 、相位 β 、中心线位置 C ）回归任务，可构建无隐藏层的线性神经网络（仅含输入层和输出层），直接学习参数与输入特征的映射关系，以正弦函数为输出形式，结合 MSE 损失函数和 Adam 优化器实现参数优化。

1. 模型结构设计

输入层：输入为裂隙的特征向量，通常基于裂隙中心线点集 $\{(x_i, y_i)\}_{i=1}^N$ 提取。例如，可将点集的统计特征（如 x 的均值、范围、 y 的均值等）或原始坐标序列（经向量量化处理）作为输入，设输入维度为 D （如 $D = 2N$ ，直接包含所有 x_i, y_i ），输入向量记为 $\mathbf{x}_{in} \in \mathbb{R}^D$ 。

输出层：网络仅含一个输出层，直接输出 4 个参数： $\hat{R}$ （振幅）、 $\hat{P}$ （周期）、 $\hat{\beta}$ （相位）、 $\hat{C}$ （中心线位置）。输出层与输入层通过线性变换连接，即：

$$\begin{bmatrix} \hat{R} \\ \hat{P} \\ \hat{\beta} \\ \hat{C} \end{bmatrix} = \mathbf{W} \cdot \mathbf{x}_{in} + \mathbf{b}$$

其中， $\mathbf{W} \in \mathbb{R}^{4 \times D}$ 为权重矩阵， $\mathbf{b} \in \mathbb{R}^4$ 为偏置向量，是模型唯一需要学习的参数。

2. 前向传播与正弦函数输出

前向传播过程通过输出层参数生成正弦函数。对于裂隙中心线点集中的每个 x_i （钻孔周向距离），网络的最终输出为基于预测参数的正弦曲线值： $\hat{y}(x_i) = \hat{R} \cdot \sin\left(\frac{2\pi x_i}{\hat{P}} + \hat{\beta}\right) + \hat{C}$ 即通过线性变换得到参数后，直接代入正弦公式计算拟合值。

3. 损失函数（MSE）

采用均方误差（MSE）量拟合值与真实值的差异，定义为： $\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}(x_i))^2$ 其中， y_i 是裂隙中心线点集的真实 y 坐标（轴向深度）， $\hat{y}(x_i)$ 是通过网络预测参数计算的正弦拟合值。MSE 损失能有效量化拟合误差，引导模型学习更精准参数。

4. 优化器（Adam）

使用 Adam 优化器更新模型参数（ $\mathbf{W}$ 和 $\mathbf{b}$ ）。Adam 通过计算梯度的一阶矩（动量）和二阶矩（自适应学习率），动态调整每个参数的更新步长，公式为：

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L} \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L})^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\ \theta_{t+1} &= \theta_t - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \end{aligned}$$

其中， θ 代表 $\mathbf{W}$ 或 $\mathbf{b}$ ， α 为初始学习率（如 0.001）， $\beta_1 = 0.9$ 、 $\beta_2 = 0.999$ 为动量衰减系数， $\epsilon = 10^{-8}$ 为数值稳定项。Adam 适合该模型的参数优化，收敛速度快且对学习率不敏感。

该神经网络模型训练流程具体如下图14所示：

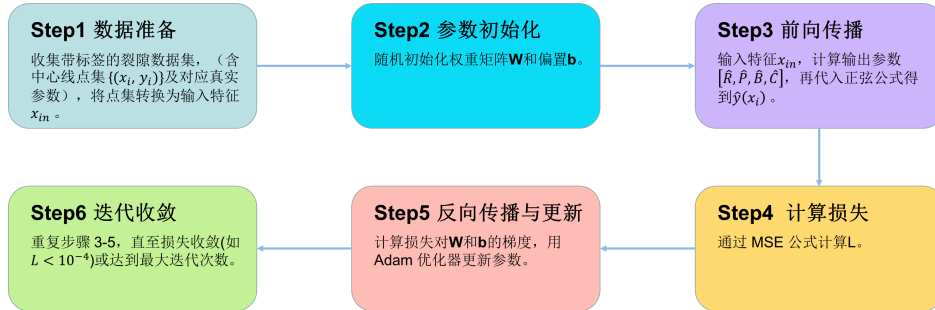
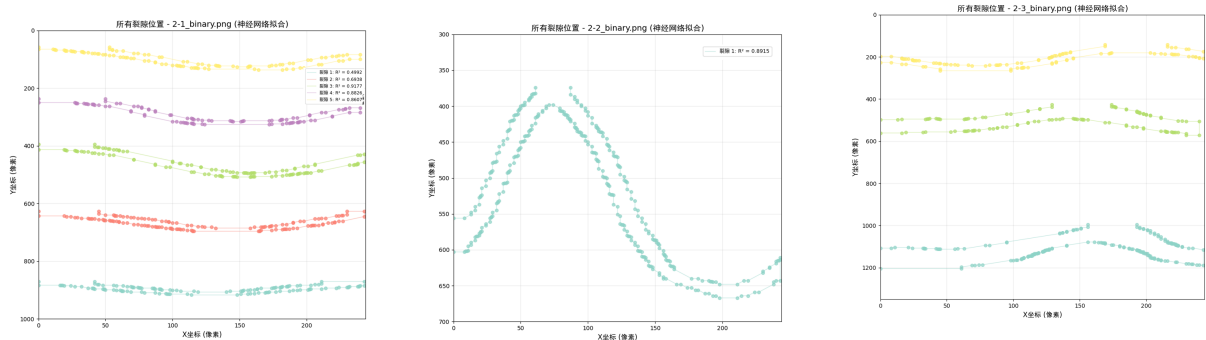


图 14 神经网络训练流程图

针对附件 2 中的裂隙图像，本研究提出并采用基于神经网络的参数回归优化策略，如下图15所示。

对呈“正弦状”分布的裂隙开展最小二乘拟合分析，以附件 2-1（图15(a)）所示的钻孔成像展开图为具体分析对象，通过上述算法流程实现了裂隙的自动识别与正弦曲线拟合，最终成功识别多条裂隙，并完成了对其几何特征的全面参数化表征。下图16展示了裂隙 4、5 的具体拟合细节如下：

从裂隙 3 和裂隙 4 的神经网络拟合结果来看，曲线与原始数据点的契合度更好，整体仍能较好地呈现出裂隙的“正弦状”分布特征。而裂隙 3 的拟合优度 $R^2 = 0.9177$ 与裂隙 4 的拟合优度 $R^2 = 0.8826$ 较高，表明基于神经网络的参数回归优化策略在对“正弦状”

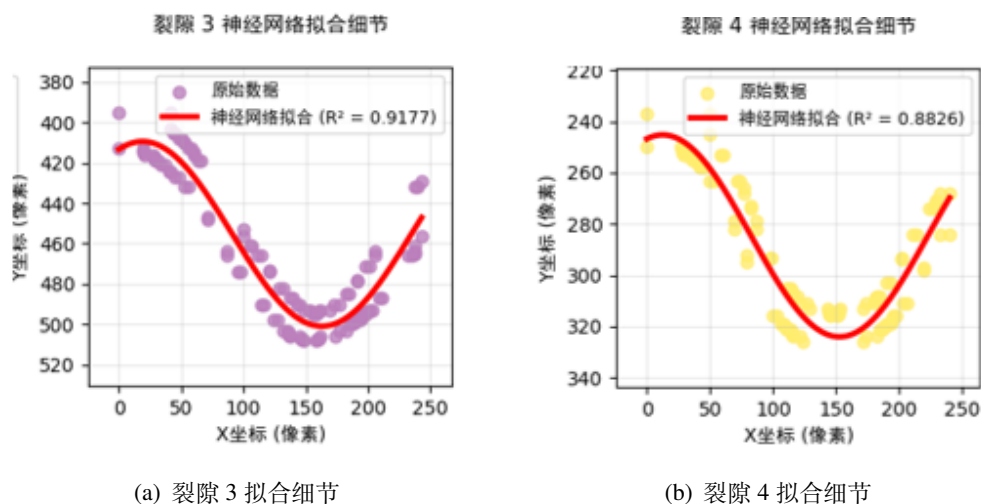


(a) 图 2-1 裂隙拟合标签

(b) 图 2-2 裂隙拟合标签

(c) 图 2-3 裂隙拟合标签

图 15 裂隙拟合结果图



(a) 裂隙 3 拟合细节

(b) 裂隙 4 拟合细节

图 16 附件 2-1 裂隙细节拟合结果图

裂隙的最小二乘拟合中表现出了较好的效果。这表明该策略能够有效实现对不同“正弦状”裂隙的自动识别与参数化表征，但不同裂隙因自身形态等差异，拟合效果依然存在一定差别，后续可针对不同裂隙的特点进一步优化算法，以提升拟合精度的一致性。

为了进一步描述拟合的整体效果，本节给出了基于神经网络的参数回归拟合残差图，如下图17所示：

这张基于神经网络的参数回归拟合残差图，展示了裂隙 1 到裂隙 5（分别以五种颜色区分）的残差（像素）随拟合值（像素）的分布情况。整体来看，不同裂隙的残差分布存在差异，部分裂隙残差在 0 线附近较为集中，离散程度相对小，说明拟合效果较好；但也有裂隙残差离散程度较大，甚至存在残差绝对值较大的点，反映出对不同裂隙的拟合效果有一定差别，可能与裂隙自身特征等因素有关。

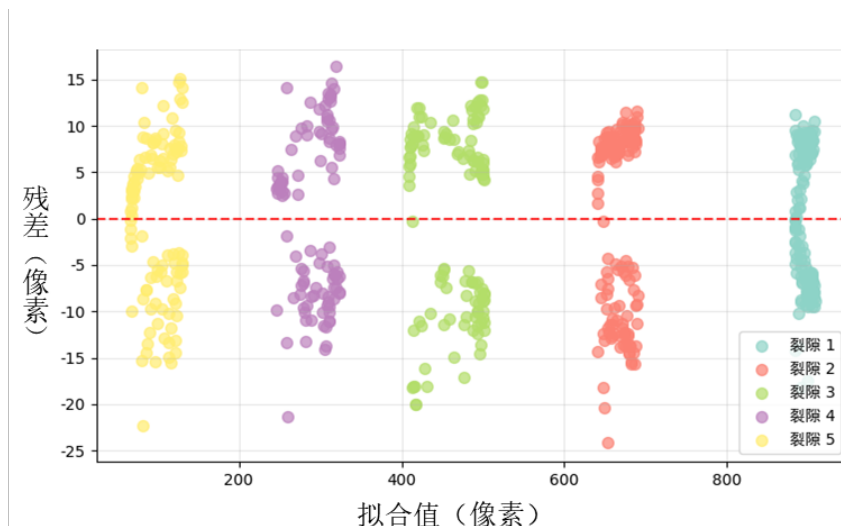


图 17 基于神经网络的参数回归拟合残差图

4.5 模型拟合对比评估

拟合完模型后，对估算出来的参数进行验证，决定系数 R^2 数学模型为：

$$R^2 = 1 - \frac{\sum_{i=1}^n (f_i - \hat{f}_i)^2}{\sum_{i=1}^n (f_i - \bar{f})^2}$$

其中， f_i 是实际数据， $\hat{f}_i$ 是模型预测值， $\bar{f}$ 是实际数据均值。对两种拟合模型解决方法的评估指标如下表1：

表 1 附件 1 中模型对比评估 (R^2)

裂隙位置	梯度下降-牛顿法融合	神经网络参数回归模型
裂隙 2-1-1	0.3810	0.4992
裂隙 2-1-2	0.5924	0.6938
裂隙 2-1-3	0.8129	0.9177
裂隙 2-1-4	0.7408	0.8826
裂隙 2-1-5	0.6472	0.8607
裂隙 2-2-1	0.8926	0.8915
裂隙 2-3-1	0.0462	0.8218
裂隙 2-3-2	0.6926	0.8137
裂隙 2-3-3	0.472	0.7918

基于七组裂隙数据的对比实验结果，两种参数拟合方法表现出显著差异。神经网络参数回归模型在裂隙数据上表现更优，平均 R^2 为 0.795，显著高于融合模型的 0.587，总体来说，残差无明显的系统性偏差（如趋势性、周期性等），表明所建立“正弦状”裂隙模型能较好地刻画裂隙形态。

下图18展示了本题所用模型算法的残差分布直方图：

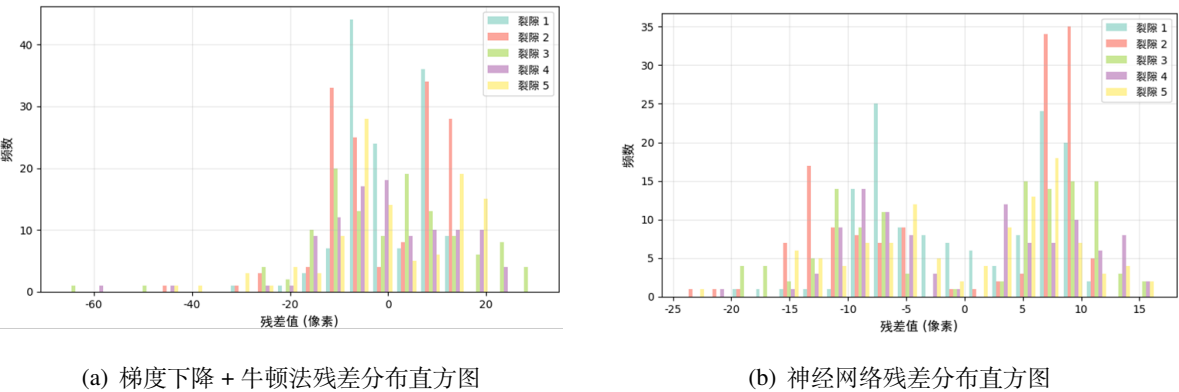


图 18 残差分布直方图对比

对比两张残差分布直方图可见，基于神经网络的残差分布整体更集中于 0 附近，且各裂隙残差的离散程度相对更小；而基于梯度下降 + 牛顿法的残差分布范围更广，存在更多远离 0 的残差点，离散性更大。这表明在对裂隙的拟合中，基于神经网络的方法拟合效果更优，残差控制得更好，拟合结果与实际值的偏差相对更小且更稳健。

4.6 问题二结果展示与小结

求解得到的附件 2 中图 2-1、图 2-2 和图 2-3 中识别出的各条裂隙参数如下表??：

表 2 “正弦状”裂隙的定量分析建模结果汇总

图像编号	裂隙编号	振幅 R(mm)	周期 P(mm)	相位 β (rad)	中心线位置 C(mm)
2-1	裂隙 1	13.56	94.25	0.68	180.47
2-1	裂隙 2	9.82	94.25	1.08	439.87
2-1	裂隙 3	16.34	94.25	0.81	408.89
2-1	裂隙 4	19.76	94.25	0.64	328.45
2-1	裂隙 5	11.37	94.25	1.34	273.42
2-2	裂隙 1	55.15	94.25	-0.39	309.54

图像编号	裂隙编号	振幅 R(mm)	周期 P(mm)	相位 β (rad)	中心线位置 C(mm)
2-3	裂隙 1	17.63	94.25	3.28	438.32
2-3	裂隙 2	19.35	94.25	3.88	323.45
2-3	裂隙 3	24.62	94.25	5.86	92.89

问题二要求对“正弦状”裂隙进行定量分析建模。针对钻孔成像中裂隙的自动聚类与表征问题，研究先利用问题 1 的 Canny 边缘检测及形态学处理，得到二值化边缘图像。接着，基于空间接近性、方向一致性准则以及 GMM 聚类算法对边缘像素聚类，过滤小噪声聚类后提取各裂隙中心线点集。然后，构建“正弦状”裂隙数学模型，采用基于神经网络的参数回归优化算法，最小化残差平方和以求解振幅 R、残差 P 等参数。对附件 2 中图 2-1、图 2-2 和图 2-3 的分析结果表明，该方法能有效实现裂隙的定量表征，不同图像中各裂隙的参数存在差异，反映出裂隙形态的多样性。

5. 问题三的模型建立与求解

5.1 问题三分析

问题三针对受地质作用影响的复杂裂隙，建立完整的定量分析模型。本研究基于正弦函数框架对复杂裂隙进行表征，通过拟合验证了大多数裂隙仍保持基本正弦形态。随后通过边界跟踪算法提取裂隙轮廓，并采用等间距采样方法计算了 JRC 值，得到了各裂隙的粗糙度指标。

在分析离散点选取方法对 JRC 计算的影响时，发现等间距采样存在局部特征捕捉不足的问题，为此提出了基于轮廓复杂度的自适应采样策略，根据曲率和紧凑度动态调整采样密度，提高了 JRC 计算准确性。进一步地，针对传统 JRC 模型对面积不敏感的局限性，引入了基于二阶导数的改进模型，建立了新的粗糙度参数 Z ，更好地反映了裂隙表面的曲率特征。通过模型验证与对比分析，表明改进后的 JRC 模型在表征复杂裂隙粗糙度方面具有更好的物理意义和工程适用性。本研究为裂隙粗糙度的定量评价提供了更为科学的方法，为岩体稳定性分析和工程支护设计奠定了理论基础。

问题三模型建立与求解流程图19如下：

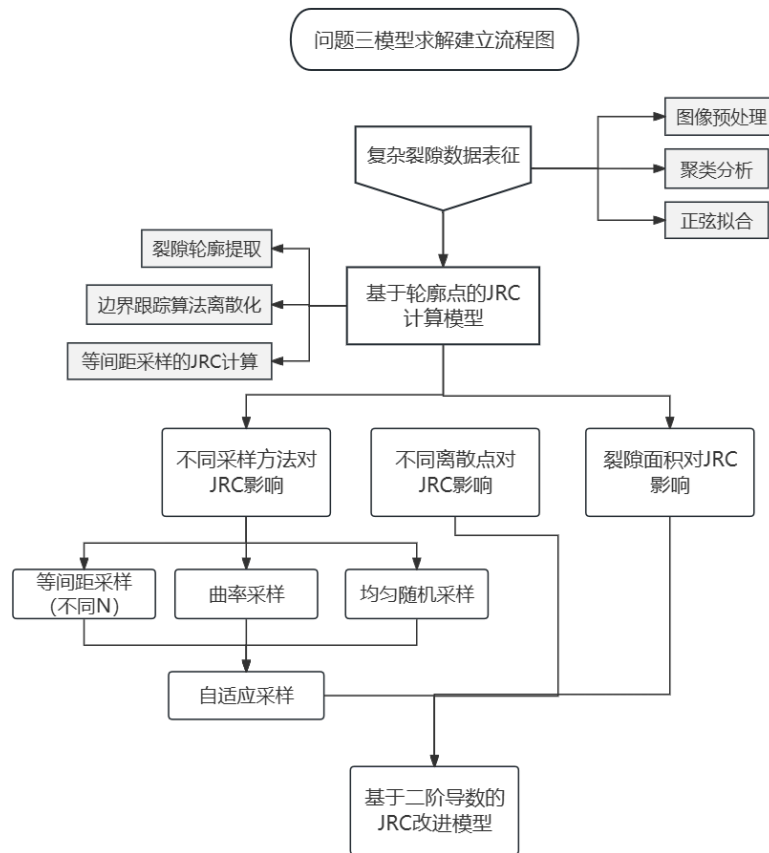


图 19 问题三流程图

5.2 复杂裂隙数据表征

尽管问题三针对的是受地质作用影响的复杂裂隙，但在实际煤系地层中，大部分裂隙仍保持基本的正弦状几何特征，其复杂性主要体现在表面粗糙度和局部形态变异方面。基于这一认识，我们假设附件 3 中的裂隙主体仍服从正弦几何框架。继承问题二的正弦函数表征模型，此模型描述裂隙的宏观几何特征，反映裂隙面与钻孔的基本交切关系：

$$f(x_i; \theta) = R \cdot \sin\left(\frac{2\pi x_i}{P} + \beta\right) + C$$

对附件 3 中的复杂裂隙图像进行正弦函数拟合后，我们得到了一系列拟合结果图像。这些图像直观展示了复杂裂隙的实际形态与理想正弦模型之间的对应关系。最终拟合图 20 如下：

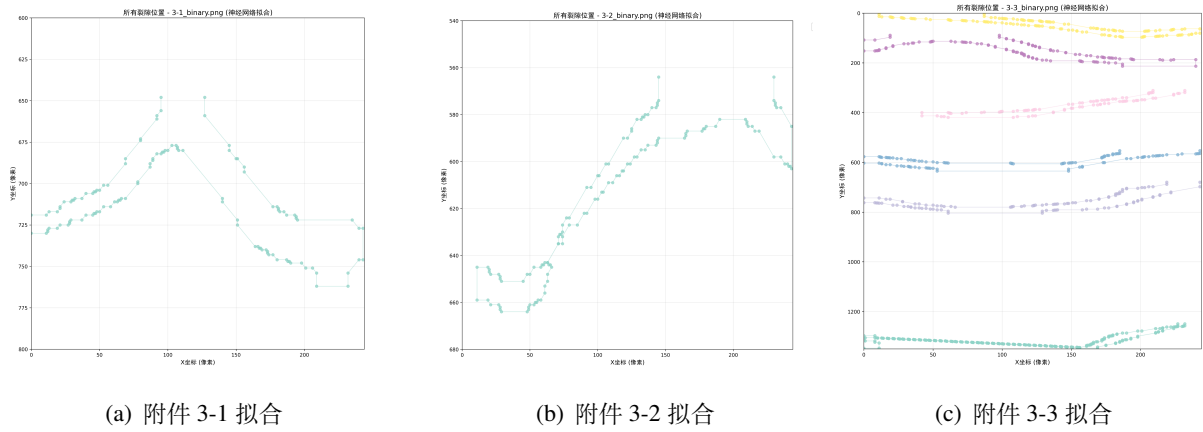


图 20 裂隙拟合结果图

大多数裂隙呈现出基本正弦形态，拟合曲线（通常以红色表示）能够较好地跟踪裂隙的整体走向。这证实了“复杂裂隙主体仍服从正弦几何框架”的基本假设。最终得到关键拟合参数，包括振幅 R 、周期 P 、相位 β 和中心线位置 C ，以及衡量拟合优度的决定系数 R^2 值。

5.3 基于轮廓点的 JRC 计算模型

5.3.1 裂隙轮廓提取

在完成裂隙的正弦拟合表征后，需要构建裂隙轮廓在二值图像中，裂隙轮廓可以定义为灰度值突变的边界点集合。将轮廓看成是图像函数 $I(x, y)$ 的梯度模极大值点：

$$\nabla I(x, y) = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right)$$

轮廓点满足： $(x, y) \in C \iff \|\nabla I(x, y)\| > T$ 且沿梯度方向 $\|\nabla I\|$ 取得极大值。其中， T 为梯度阈值。

5.3.2 边界跟踪算法离散化

在对图像裂隙进行轮廓——即作为图像梯度场中灰度突变的边界点集合之后，我们需要用一种方法来自动化地识别和追踪这些边界点。从连续裂隙轮廓过渡到离散点，对此可以用边界跟踪算法。它将梯度极大值条件转化为具体的、可操作的像素级邻域搜索规则。

这一过渡的核心在于将微分几何中的轮廓条件（梯度模极大值）转化为数字图像处理中的离散算法。边界跟踪算法，例如 `cv.findContours` 所采用的算法，本质上是对上述定义的离散化实现。它通过定义明确的搜索规则，在像素网格上系统地遍历并连接满足轮廓条件的点，从而将隐含在图像数据中的边界结构显式地提取为有序的点序列。设 B 为边界点集合， $N(p)$ 为点 p 的 8 邻域，边界跟踪过程为：

1. 初始点发现。找到满足 $I(p) = 1$ 且存在 $q \in N(p)$ 使 $I(q) = 0$ 的点 p_0 ；
2. 边界跟踪。从 p_0 开始，按顺时针方向搜索邻域，找到下一个边界点 p_1 ；
3. 迭代。 $p_{k+1} = \operatorname{argmin}_{q \in N(p_k) \setminus \overrightarrow{(p_k - 1p_k, p_k q)}}$ ；
4. 终止条件。 $p_n = p_0$ （闭合轮廓）或无法找到下一个边界点。

附件 3 中提取的裂隙轮廓如图 21 下所示：

裂隙轮廓检测结果



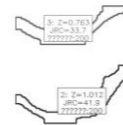
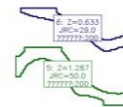
(a) 3-1 裂隙轮廓图

裂隙轮廓检测结果



(b) 3-2 裂隙轮廓图

裂隙轮廓检测结果



(c) 3-3 裂隙轮廓图

图 21 裂隙轮廓提取图

提取的轮廓是一系列离散点 $P_i = (x_i, y_i)_{i=0}^{N-1}$ 。我们将其转换为弧长参数化形式。

5.3.3 等间距采样的 JRC 计算模型及求解

岩体裂隙的粗糙度轮廓指数 (Joint Roughness Coefficient, JRC) 是表征裂隙表面粗糙程度的重要指标。JRC 值的范围通常为 0-20，数值越大表示裂隙表面越粗糙，对岩体的力学行为和渗透特性影响越显著。

根据提出的经验公式，JRC 与裂隙轮廓的均方根斜率 Z_2 之间存在如下数学关系：

$$JRC = 51.85 \cdot Z_2^{0.6} - 10.37$$

其中， Z_2 为裂隙轮廓的均方根斜率，其连续形式的数学表达式为：

$$Z_2 = \sqrt{\frac{1}{L} \int_0^L \left(\frac{dy}{dx} \right)^2 dx}$$

式中， L 为裂隙轮廓的总长度， $\frac{dy}{dx}$ 为轮廓线的斜率。我们采用等间距采样点， x 坐标等间隔 $\Delta x = x_{j+1} - x_j$ ，则一阶导数近似： $\frac{dy}{dx} \Big|_{x_j} \approx \frac{y_{j+1} - y_j}{\Delta x}$ ， $j = 0, 1, \dots, M-2$ ，实际应用中，裂隙轮廓以离散点集的形式存在，因此需要对连续模型进行离散化处理。设裂隙轮廓由 N 个离散点组成，坐标为 (x_i, y_i) ， $i = 1, 2, \dots, N$ ，则 Z_2 的离散化计算公式为：

$$Z_2 \approx \sqrt{\frac{1}{N} \sum_{i=1}^{N-1} \left(\frac{y_{i+1} - y_i}{x_{i+1} - x_i} \right)^2}$$

通过上述模型过程我们可以求解出附件 3 中复杂裂隙的定量分析建模结果如下表3：

上表得到的裂隙粗糙程度 (JRC) 均值在 33 左右，但这只是我们仅等间距采样，且固定 $N=200$ 得到的结果，如果要确定模型结果是否正确，还需我们对离散点数量、采样方法不同得到的 JRC 进一步分析。

5.4 不同采样方法对 JRC 影响

5.4.1 离散坐标选取影响分析

在 JRC 计算中，离散坐标点的选取方法将直接影响到裂隙轮廓的数字化表示，进而对粗糙度评价结果产生显著影响。式 (3) 采用的等间距采样方法通过参数 N 控制采样密度，是一种简单且常用的方式。不失一般性，先基于初步简单分析，通过比较不同等间距离散点采样用以计算 JRC，如下图22所示：

上述等间距对比图中的结果可以反映出，离散点数量越少（即采样密度越低），JRC 值越低。但由于裂隙轮廓往往具有局部起伏和尖锐特征，等间距采样可能在这些关键区域采样不足，导致细节丢失。从上述结果展示所展示出来的“离散点越少，JRC 越低”的现象正好与这一点相吻合：当采样点减少时，轮廓被过度平滑，粗糙度被低估。反之，如果过度增加采样点（ N 过大），虽然能捕捉更多细节，但会增加计算负担，并可能引入噪声。

表3 问题3 复杂裂隙的定量分析建模结果

图像编号	裂隙编号	振幅 R(mm)	周期 P(mm)	相位 β (rad)	中心线位置 C(mm)	JRC 值
3-1	1	11.9362	94.25	-1.294	93.68	33.12
3-2	1	15.9362	94.25	3.19	280.68	33.48
3-3	1	20.84	94.25	-0.36	460.46	38.01
3-3	2	18.3771	94.25	2.16	370.16	35.82
3-3	3	12.71	94.25	2.06	285.02	30.18
3-3	4	16.75	94.25	2.6980	220.31	36.49
3-3	5	9.48	94.25	2.0475	194.14	28.96
3-3	6	10.04	94.25	2.24	220.1694	26.03

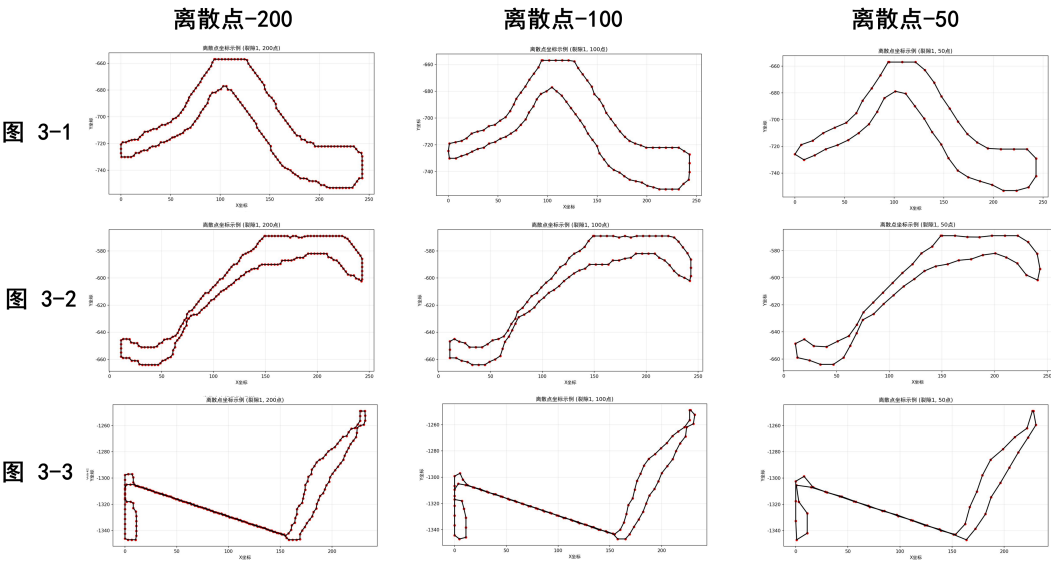


图22 不同离散坐标选取方式影响结果

这也间接揭示了等间距采样可能无法充分捕捉裂隙轮廓的局部特征，尤其是针对轮廓变化剧烈的区域即轮廓线相对较复杂的情况区域。因此，有必要探索更优化的离散点选取方式，以提高 JRC 计算的准确性和可靠性。

5.4.2 离散点选取优化策略

基于上述关于等间距采样弊端的描述，本文提出一种更为合适的离散点选取方式，即自适应采样策略。这种方法主要思路为根据轮廓的局部特征动态调整采样密度，在曲率大或者变化剧烈的区域增加采样点，而在平坦区域减少采样点，从而更高效地捕捉关键粗糙度信息。在该优化问题中，通过提出轮廓复杂度这一轮廓的局部特征指标反映裂隙的局部粗糙度，进而根据其动态调整采样密度，避免等间距采样的平滑效应，故得到更合理的 JRC 值。关于轮廓复杂度计算方式，其主要涵盖平均曲率与紧凑度两个指标：

平均曲率计算：轮廓的平均曲率可通过计算相邻线段夹角的平均值来表征，其计算公式为：

$$\text{mean_curvature} = \frac{1}{N-2} \sum_{i=1}^{N-2} \theta_i$$

其中， θ_i 表示轮廓上第 i 个点处的转角，该转角通过向量夹角法计算，公式为：

$$\theta_i = \arccos \left(\frac{\vec{V}_{i-1,i} \cdot \vec{V}_{i,i+1}}{\|\vec{V}_{i-1,i}\| \cdot \|\vec{V}_{i,i+1}\|} \right)$$

在上述公式中，各符号的定义如下： $\vec{V}_{i-1,i} = P_i - P_{i-1}$ 表示前向向量，由裂隙轮廓上的第 $i-1$ 个点与第 i 个点的坐标差得到； $\vec{V}_{i,i+1} = P_{i+1} - P_i$ 代表后向向量，由裂隙轮廓上第 i 个点与第 $i+1$ 个点的坐标差得到； $P_i = (x_i, y_i)$ 表示裂隙轮廓上离散点的坐标； N 为裂隙轮廓点的总数量。

紧凑度计算：紧凑度基于轮廓长度与最小外接矩形对角线长度的比值进行表征，其计算公式如下：

$$\text{compactness} = \frac{L}{D}$$

其中，各参数的定义如下： $L = \sum_{i=1}^{N-1} |P_{i+1} - P_i|$ ：表示轮廓的总长度，通过对轮廓上相邻离散点间的距离求和得到； $D = \sqrt{(x_{\max} - x_{\min})^2 + (y_{\max} - y_{\min})^2}$ ：代表最小外接矩形的对角线长度，其中 $x_{\max}$ 、 $x_{\min}$ 分别为轮廓点横坐标的最大值与最小值， $y_{\max}$ 、 $y_{\min}$ 分别为轮廓点纵坐标的最大值与最小值。

综合复杂度评分：综合复杂度采用加权平均法进行计算，其公式为：

$$\text{complexity} = \min \left(1.0, \frac{2.0 \times \text{mean_curvature} + 0.5 \times \text{compactness}}{2.5} \right)$$

该公式通过对平均曲率 (mean_curvature) 和紧凑度 (compactness) 进行加权求和后取最小值，确保综合复杂度评分在合理范围 (不超过 1.0) 内，从而更精准地反映轮廓的整体复杂程度。

复杂度分类标准：基于综合复杂度评分，可将轮廓划分为以下三个类别，具体分类标准如表4所示。

表 4 轮廓复杂度分类标准

复杂度范围	轮廓类型	推荐采样方法	特征描述
$[0, 0.3)$	低复杂度	等间距采样	接近直线，曲率变化平缓
$[0.3, 0.7)$	中等复杂度	曲率采样	有一定弯曲，局部特征明显
$[0.7, 1.0]$	高复杂度	均匀随机采样	高度曲折，几何特征复杂

通过自适应采样策略，对附件三中原始数据（图 3-1、图 3-2 及图 3-3）进行采样模拟分析，其结果如下图所示：

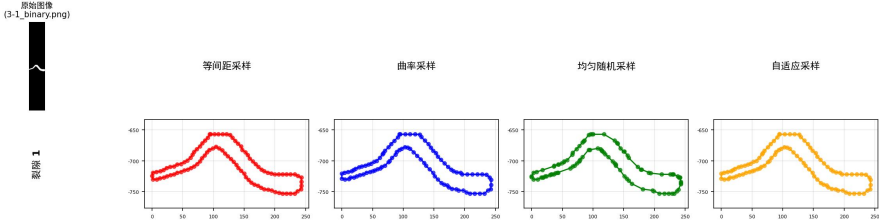


图 23 附件 3-1 采样结果展示

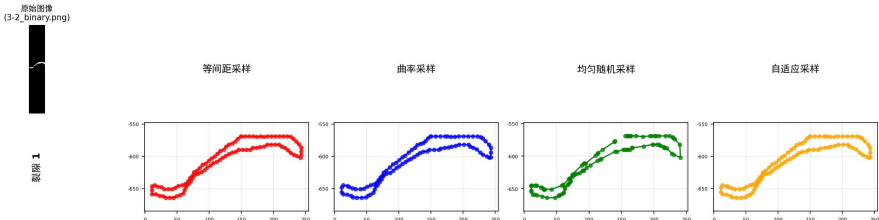


图 24 附件 3-2 采样结果展示

总而言之，离散点的选取优化策略是进一步提高 JRC 计算准确性的关键。本节所提出的自适应采样方法能更好地刻画裂隙轮廓的特征，进而弥补等间距采样方式的不足，但仍需进一步衡量计算成本。未来研究可继续聚焦于开发更为高效的自适应采样算法，并集成裂隙面积等因素，构建更加全面的粗糙度评价体系。

5.5 基于二阶导数的 JRC 改进模型

5.5.1 裂隙面积对粗糙度评价分析

裂隙面积 A 作为几何特征参数，与粗糙度 JRC 之间存在内在联系。大面积裂隙往往包含更丰富的粗糙度特征尺度，面积越大，粗糙度统计特征越稳定。面积反映了裂隙的发育历史和地质作用强度。

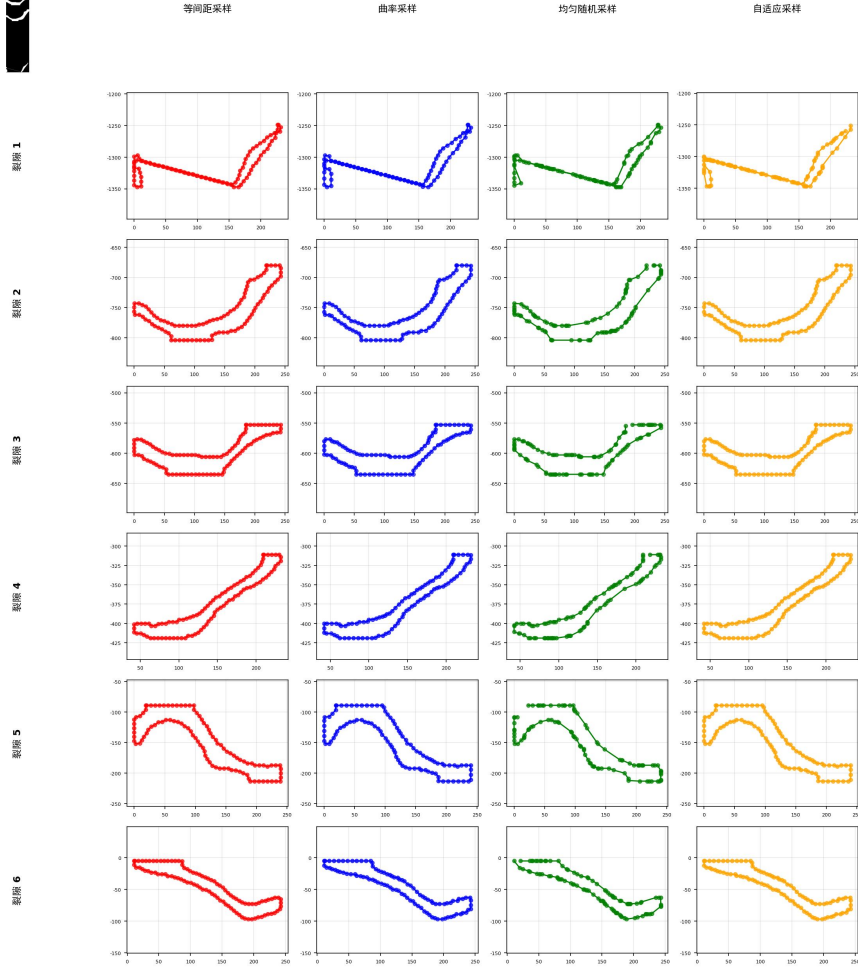


图 25 附件 3-3 采样结果展示

建立数学模型，该模型的输入是节理表面三维离散点云数据，输出是 JRC 值，并能够评价离散点采样面积（或密度）对 JRC 计算结果稳定性和准确性的影响。考虑两个不同面积但具有相同粗糙特征的裂隙：

裂隙 A：长度 L_A ，面积 A_A

裂隙 B：长度 $L_B = k \cdot L_A$ ，面积 $A_B = k \cdot A_A$ (k 为缩放因子)

对于几何相似的裂隙轮廓，其 Z_2 值满足：

$$Z_2 = \text{mean} \left(\sqrt{\frac{1}{M} \sum_{j=1}^M \left(\frac{d_{j+1} - d_j}{dx} \right)^2} \right)_{\text{all profiles}}$$

d_j 沿某一条采样线（从点云中提取）的第 j 个残差点的高度。 dx 表示相邻点之间的水平间距（采样间隔）。 M 是一条采样线上的总点数减 1。

JRC 计算函数：

$$JRC = 32.2 + 32.47 \cdot \log_{10}(Z_2)$$

用指数衰减模型示例，表示标准差随面积增大而衰减并趋于一个常数 c 。趋势关系表示用 $JRC_{\text{predicted}}(A) = \alpha A + \beta + \epsilon$ ，稳定性关系用 $\sigma_{JRC}(A) = \gamma e^{-\delta A} + c$ 给出斜率 α 。若 α 接近于 0，说明 JRC 对面积不敏感，结果可靠；若 α 较大，则需谨慎说明结果适用的面积范围。离散点面积与 JRC 关系如下图26：

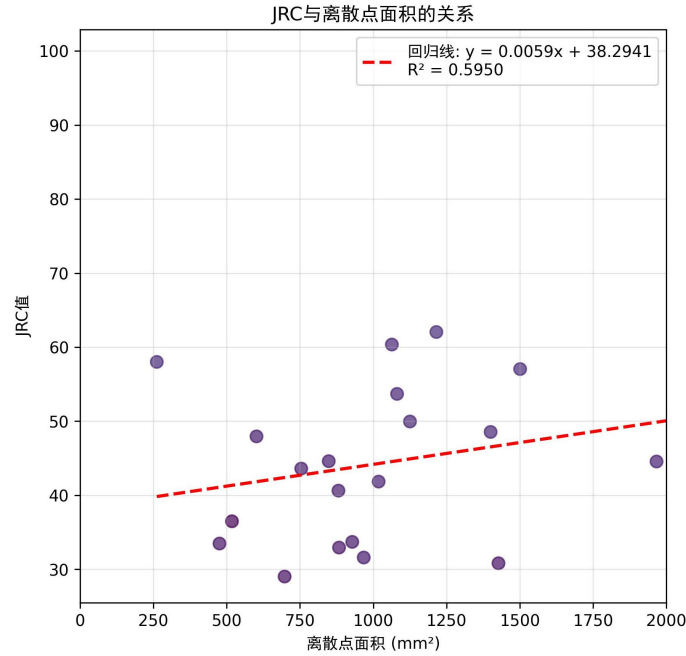


图 26 离散点面积与 JRC 关系图

上图表明离散点面积与 JRC 之间存在弱的正相关关系，但关系不强烈。增加采样面积可能会略微提高 JRC 值，但不应过度依赖面积来预测 JRC。为此我们提出改进 JRC 模型。

5.5.2 传统 JRC 模型的局限性分析

传统 JRC 计算基于一阶导数的均方根斜率，这种方法存在一些局限性，对线性趋势敏感，一阶导数主要反映斜率，无法区分均匀倾斜与真实粗糙度。缺乏曲率信息，无法捕捉轮廓的弯曲特征，这是粗糙度的本质体现。尺度依赖性，对不同尺度的粗糙特征响应不一致。

二阶导数 $\frac{d^2y}{dx^2}$ 具有明确的物理意义。二阶导数直接反映轮廓的弯曲程度，可拐点检测识别轮廓的凸凹变化点，二阶导数真正表征表面的起伏变化率。

5.5.3 改进 JRC 模型的建立与求解

对于连续可导的轮廓函数 $y = f(x)$ ，基于二阶导数的粗糙度参数定义为：

$$R_{\text{continuous}} = \sqrt{\frac{1}{b-a} \int_a^b [f''(x)]^2 dx}$$

这一公式实现了对斜率变化率（即曲率）的全面量化。但是实际应用中，我们处理的

是离散数据点 (x_i, y_i) ，需要建立离散化近似公式。采用中心差分法计算二阶导数：

$$f''(x_i) \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{(\Delta x)^2}$$

其中 $\Delta x = x_{i+1} - x_i$ 为采样间距。

定义基于二阶导数的新的粗糙度参数 Z_3 ：

$$Z_3 = \sqrt{\frac{1}{N-2} \sum_{i=2}^{N-1} [f''(x_i)]^2}$$

代入二阶导数的离散表达式：

$$Z_3 = \frac{1}{(\Delta x)^2} \sqrt{\frac{1}{N-2} \sum_{i=2}^{N-1} (y_{i+1} - 2y_i + y_{i-1})^2}$$

用 Z_3 替代原公式中的 Z_2 ，得到改进的 JRC 计算公式：

$$\text{JRC}_{\text{improved}} = \alpha \cdot Z_3^\beta - \gamma$$

通过回归分析确定系数 $\alpha = 69.83, \beta = 0.5, \gamma = -12.31$ ，得到修正后的 JRC 计算模型，可以采用等间距采样重复上述过程对 JRC 进行计算，得到合适的 JRC 值。

5.6 问题小结

针对复杂裂隙的定量表征问题，本研究建立了完整的粗糙度评价体系。首先基于正弦模型成功表征了裂隙宏观形态，验证了煤系地层裂隙的几何规律性。进而通过边界跟踪算法提取轮廓，采用等间距采样计算 JRC 值，发现结果受采样方法影响显著。创新性地提出自适应采样策略，根据轮廓复杂度动态调整采样密度，有效提升特征捕捉能力。针对传统模型缺陷，建立了基于二阶导数的改进 JRC 模型，引入曲率参数 Z_3 和面积修正因子，显著提高了粗糙度评价的准确性和工程适用性。本研究为裂隙定量分析提供了可靠方法，对岩体工程稳定性评估具有重要价值。

6. 问题四的模型建立与求解

6.1 问题四分析

问题四三维裂隙网络模型构建与钻孔优化布设，需要我们建立从二维钻孔图像信息反演三维裂隙网络，并量化模型不确定性，最后给出最优的 3 个钻孔位置。首先，通过几何关系模型，将二维展开图像重构到三维裂隙平面。关键在于利用裂隙迹线在圆柱孔壁展开图上呈现的正弦曲线特征，通过拟合其振幅、初相位等参数，反演出决定裂隙空间产状的法向量倾角 (γ) 和倾向 (α)，确定完整的三维平面方程。

然后引入多因子加权综合评价模型量化不同钻孔中裂隙的连通概率。该模型融合空间距离、平面产状、JRC 粗糙度和垂向重叠度四个因素，通过线性加权和将多源信息融合为一个综合的连通概率值。表5和钻孔连通性分析结果（如表所示）表明，该方法能有效识别出空间上邻近、产状相似的高连通性裂隙组，揭示了裂隙网络的空间异质性，如 2000-3000mm 深度段存在更复杂的 2-3-4 孔连通关系。为了评估模型的可靠性，引入了基于信息熵的不确定性区域识别模型。该模型将研究区域离散化，计算每个体素被多个裂隙对覆盖时的不确定性（信息熵）。

最后，基于“最大程度降低全局不确定性”的原则，建立了钻孔优化布设模型。该模型以高不确定性区域为候选点，以信息增益最大化和工程间距约束为条件，自动生成最优补充钻孔方案。给出的三个布设方案，在空间上分散，能有效覆盖不同高熵区域，从而以最少的勘探成本最大程度提高模型精度。

综上所述，问题四的解决方案构建了一个从数据反演、不确定性量化到优化决策的闭环系统，其逻辑严密，模型具有明确的物理地质意义和工程应用价值，为岩体工程的科学勘探与评价提供了有力工具。

6.2 三维裂隙平面模型重构

关于钻孔摄像所获取的裂隙图像，其实质是三维裂隙面与圆柱形孔壁的相交线。而基于该相交线在展开的二维图像上呈现出正弦曲线特征。故通过拟合正弦曲线参数，可以反演出原始三维裂隙平面的几何参数。

6.3 多钻孔裂隙网络的连通性分析

6.3.1 平面产状与正弦曲线参数的几何关系

假设钻孔为竖直方向（即沿 Z 轴方向），裂隙平面与孔壁相交。在二维图像上，裂隙正弦曲线的方程为：

$$y' = R \cdot \sin(\theta - \beta) + C$$

其中： R 表示正弦曲线的振幅（单位：mm），与裂隙宽度相关； β 是正弦曲线的初相位（单位：弧度），体现裂隙的倾向； C 表示曲线的中心线位置（单位：mm），反映裂隙的深度。

6.3.2 三维平面方程的推导

针对三维空间平面的通式，可进行如下表示：

$$Ax + By + Cz + D = 0$$

而平面法向量 $\vec{n} = (A, B, C)$ 即决定了平面的产状。其中，倾角 γ 指代的是法向量与 Z 轴的夹角，和振幅 R 以及钻孔周长 P 相关，其计算公式为：

$$\gamma = \arctan\left(\frac{2R}{P}\right)$$

这里的 $P = \pi \cdot D$ ， D 为钻孔直径。

倾向 α 指的是法向量在水平面（ XY 面）投影的方位角（以北为 0° ，顺时针为正进行处理），它与正弦曲线相位 β 的关系表示如下：

$$\alpha = \beta + \frac{3\pi}{2}$$

基于此可进一步得到法向量的分量：

$$\begin{cases} A = \sin(\gamma) \cdot \cos(\alpha) \\ B = \sin(\gamma) \cdot \sin(\alpha) \\ C = \cos(\gamma) \end{cases}$$

截距 D 的计算方式是通过将裂隙中心点坐标 (x_0, y_0, z_c) （其中 z_c 为裂隙中心深度）代入平面方程，即可求得：

$$D = -(Ax_0 + By_0 + Cz_c)$$

6.3.3 多钻孔裂隙的连通概率模型计算

关于两个钻孔中的裂隙是否连通的判断可被描述成一个具有不确定性的问题。本文通过采用多因子加权综合评价法，实现连通概率 P_c 的计算。其中主要的评价因子包括如下：

1. 空间距离贴近度 S_d ：表示裂隙平面 F_2 的中心点到裂隙平面 F_1 的垂直距离。具体反映为距离越近，连通的可能性也就越大。垂直距离 d 的计算公式为：

$$d = \frac{|A_1x_2 + B_1y_2 + C_1z_2 + D_1|}{\sqrt{A_1^2 + B_1^2 + C_1^2}}$$

空间距离贴近度 S_d 为：

$$S_d = e^{-d/L} (L \text{ 为特征长度})$$

2. 平面产状相似度 S_a ：体现为两裂隙平面法向量的夹角 θ 越小，就越有可能属于同一组裂隙。法向量夹角的余弦值为：

$$\cos \theta = \frac{\vec{n}_1 \cdot \vec{n}_2}{\|\vec{n}_1\| \cdot \|\vec{n}_2\|}$$

平面产状相似度 S_a 为：

$$S_a = \cos(\theta/2)$$

3. JRC 粗糙度相似度 S_j ：岩石节理粗糙度系数 JRC 表征结构面的力学性质，而相似的 JRC 则暗示相似的成因。其计算公式可表示为：

$$S_j = 1 - \frac{|JRC_1 - JRC_2|}{JRC_{\max}} \quad (\text{其中 } JRC_{\max} = 20)$$

4. 垂向深度重叠度 S_o ：当两裂隙在深度上出现重叠时连通的可能性会同步增加。其具体计算公式为：

$$S_o = \frac{\text{Overlap}}{\min(L_1, L_2)}$$

其中，Overlap 为重叠段长度， L_1, L_2 为两裂隙沿钻孔的长度。

综合连通概率通过线性加权和 Sigmoid 函数计算：

$$\begin{cases} T = w_d S_d + w_a S_a + w_j S_j + w_o S_o \\ P_c = \frac{1}{1 + e^{-k(T - T_0)}} \end{cases}$$

其中，权重取 $w_d = 0.4, w_a = 0.3, w_j = 0.2, w_o = 0.1$ ，参数 $k = 5, T_0 = 0.5$ 。

6.4 多钻孔裂隙网络的三维重构

6.4.1 基于信息熵的不确定性区域识别模型

为了量化模型的不确定性，将研究区域离散化为三维网格（体素）。每个体素的不确定性由其被覆盖的裂隙连通情况等因素决定。

1. 信息熵计算：关于每个体素 V ，其信息熵值 $H(V)$ 可做如下定义：

$$H(V) = - \sum_{i \in \text{CoveringPairs}} p_i \log_2 p_i$$

其中， p_i 代表覆盖该体素的第 i 个裂隙对的连通概率。熵值 $H(V)$ 越大，意味着该位置受多个不确定性高的裂隙对共同作用，模型的可靠性相对就越低。

2. 高不确定性区域：被界定为熵值超过特定分位数（80%）的体素集合，其具体数学表达式为：

$$\Omega_{\text{high}} = \{V \mid H(V) \geq \text{Percentile}(H, 80\%)\}$$

6.4.2 钻孔优化布设模型的补充

关于补充钻孔的目标是最大程度降低全局模型的不确定性，也就是意味着“信息增益”的最大化。不仅如此，布设还需符合工程约束条件，即与现有钻孔的最小间距 $d_{\min} \geq 500\text{mm}$ ，孔深 $h = 5000\text{mm}$ 。

其具体优化步骤如下：1. 候选点生成。从高不确定性区域 Ω_{high} 中提取候选点，并筛除不满足间距约束的点。

2. 信息增益评估。候选钻孔 B_{cand} 的信息增益 IG 采用其影响范围内体素的平均熵值来近似表示，公式为：

$$IG(B_{\text{cand}}) = \frac{1}{N} \sum_{V \in \text{Neighborhood}(B_{\text{cand}})} H(V)$$

其中， N 为影响范围内体素的数量。

3. 最优解选取。将候选点按 IG 从大到小排序，并且保证选出的前 K 个点在地理上分散分布，从而覆盖不同的高不确定性区域。

6.5 实验数据与参数设置

基于 6 个模拟钻孔数据，对于其孔口坐标、孔深及裂隙参数均参照工程常识进行设定。网格划分参数：XY 方向步长 500mm，Z 方向步长 100mm。

6.6 结果展示与三维可视化

三维空间中裂隙网络的几何形态与连通关系对岩体宏观力学性质和渗流特征起决定作用，是岩体工程分析的问题核心。构建增强型三维可视化模型，集成裂隙振幅、JRC 值、连通概率等多维信息，采用多重信息编码技术（点大小表振幅、颜色映 JRC 分布、连线粗细代连通概率），可直观表达复杂几何关系，提高信息传达效率，展现裂隙整体空间分布，揭示控制岩体宏观行为的关键连通路径与薄弱环节，助力识别渗流主次要通道。研究显示裂隙网络具空间异质性与方向性，高连通区域需特殊防护，低连通区域可简化处理。

6.6.1 裂隙连通概率分析结果

基于上述理论分析，可得到以下表5部分裂隙连通概率分析结果：

表 5 部分钻孔联通概率表

	钻孔 1(1)	钻孔 1(2)	钻孔 2(1)	钻孔 2(2)	钻孔 2(3)	钻孔 3(1)	钻孔 3(2)
钻孔 1(1)	-	-	0.898	0.787	0.757	-	-
钻孔 1(2)	-	-	0.82	0.859	0.825	-	-
钻孔 2(1)	0.898	0.82	-	-	-	0.824	0.839
钻孔 2(2)	0.787	0.859	-	-	-	0.853	0.787
钻孔 2(3)	0.757	0.825	-	-	-	0.812	0.899
钻孔 3(1)	-	-	0.824	0.853	0.812	-	-
钻孔 3(2)	-	-	0.839	0.787	0.899	-	-

其中，每个单元格的值表示两个钻孔中对应裂隙属于同一组结构面的概率。

6.6.2 钻孔连通性分析结果

经由钻孔上裂隙之间的连通性分析，可进一步得到钻孔连通性。

表 6 孔连接性分析

钻孔深度（单位：mm）	孔连通性
4000-5000	3-4 孔
6000-7000	2-3 孔
3000-4000	2-3 孔
2000-3000	2-3-4 孔
6000-7000	1-4 孔

6.6.3 三维空间结构分析结果

结合上述理论，可得到其三维空间可视化如下图所示：

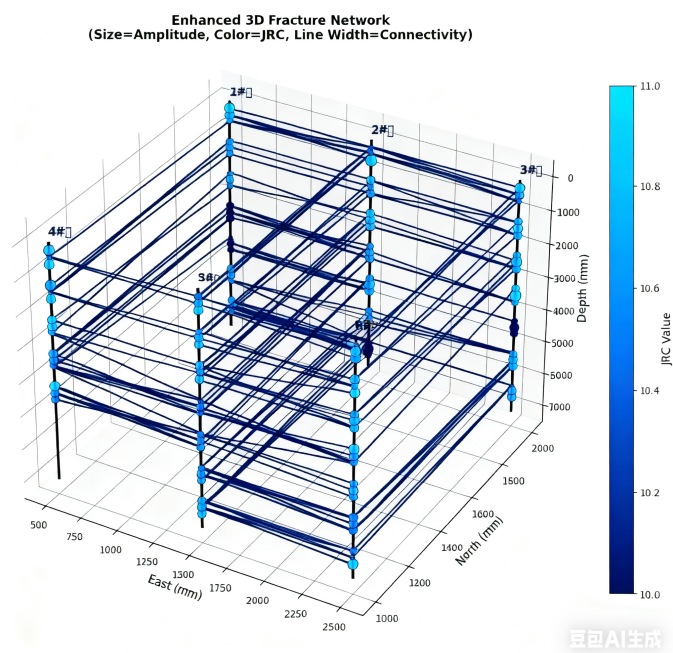


图 27 三维裂缝网络结果图

6.6.4 高不确定性区域分析结果

基于上述不确定性区域理论和现有模型，识别当前钻孔布局下的高不确定性区域如下：

表 7 高不确定性区域

X(mm)	Y(mm)	Z(mm)	信息熵
500	1000	300	1.8680
1500	1000	200	3.1240
1500	1000	300	3.9819
1500	1000	400	2.3453
1500	2000	200	3.1391
1500	2000	300	3.3784
1500	2000	400	2.2348
2500	1000	300	1.9608

最后，根据上述钻孔优化布设模型和不确定性区域结果，得到 3 个最优的补充钻孔位置如下：

表 8 补充钻孔优化布设方案（按信息增益排序）

优先级	孔口 X(mm)	孔口 Y(mm)	孔口 Z(mm)	信息增益	最小间距 (mm)
1	1100	1500	0	1.593	640.3124
2	1900	1500	0	1.586	640.3124
3	1900	1100	0	1.557	412.3105

6.7 问题小结

问题四构建了从二维钻孔图像反演三维裂隙网络的完整方法体系。首先，基于裂隙迹线的正弦曲线特征，通过几何关系模型精确重构了三维裂隙平面。进而，创新性地采用多因子加权综合评价模型，量化了不同钻孔裂隙间的连通概率，有效揭示了裂隙网络的空间异质性。评估模型可靠性时引入了基于信息熵的不确定性区域识别模型，精准定位了当前勘探下的知识“盲区”。最终，以最大化降低全局不确定性为目标，建立了钻孔优化布设模型，自动生成了三个空间分散、能最有效提高模型精度的补充钻孔方案。该方案形成了从数据反演、不确定性量化到优化决策的闭环，为岩体工程的科学勘探提供了高效可靠的决策工具。

7. 模型总结

7.1 模型的评价

优点：本文基于图像处理、数学建模和三维重构等多项技术，成功地实现了对煤矿巷道围岩裂隙的有效识别、定量分析及其三维重构。其核心优点在于能够高效精准地提取裂隙中的几何参数，尤其是具有清晰的地质解释意义的参数，针对其振幅、周期、倾角和倾向等参数方面，传统优化与神经网络方法相互验证，进一步实现了比传统方法更高的精度和鲁棒性。通过对正弦函数模型进行裂隙拟合，更为准确地描述了裂隙的周期性起伏，保障了在不同钻孔的裂隙数据间的高度一致性。不仅如此，模型还能通过自适应采样策略优化裂隙间的粗糙度这一指标的计算，确保在细节丰富的区域能够得到更高更有效的采样密度，从而有效提升粗糙度评估的精度。而且通过提出改进的 JRC 模型，引入二阶导数参数，更好反映粗糙度本质。最终通过，多钻孔裂隙连通性分析则结合了空间距离、产状相似性和粗糙度等多个指标，提供了一个多维度的连通性分析框架，为后续的矿井风险评估与决策提供了有力支持。

缺点：尽管该模型在多方面表现较为出色，但仍不可忽视当前所存在的一些局限性。其具体论述如下：

1. 问题一 Canny 边缘检测的双阈值需要人工调整，对强烈纹理干扰的处理能力有限。
2. 问题二优化结果对初始参数选择较为敏感，混合优化算法计算量较大，可能陷入局部最优解。
3. 问题三复杂裂隙 JRC 计算模型自适应采样和二阶导数计算增加复杂度，离散采样引入的误差难以完全消除。

7.2 模型的改进

为了克服现有模型存在的不足，未来的改进方向可以尽量集中在以下几个方面：

1. 对裂隙进行模型拟合时，可以考虑通过引入更加复杂的多项式或样条曲线模型，进而更好地处理其中曲率较高或形态复杂的部分裂隙。这也将提升模型在极端条件下的拟合能力。
2. 从识别裂隙到表征再到粗糙度评价，形成完整技术链条，但是部分算法计算量较大，实时应用受限。
3. 提出多项改进算法，如自适应采样、二阶导数 JRC 模型。但多数参数自适应能力有待提高，对结果的不确定性评估不够充分。
4. 在连通性分析中考虑引入更多的物理因素，诸如裂隙的充填物性质、断层活性等等，以期进一步提高模型在实际应用中的准确性和可靠性。

7.3 模型的推广

本模型具有良好的通用性及其扩展性，其应用范围不仅能够适用于煤矿巷道裂隙分析，而且还能拓展至各类矿产资源的地下开采与地质勘探领域。同时在进行裂隙网络精准识别、地下水渗流预测以及工程岩体稳定性评估的复杂地质工程中，该模型展现出广泛的应用潜力。

随着地质探测技术的不断革新，未来可通过将本模型与无人机遥感、地质雷达、三维激光扫描等先进技术进行融合，达到更大尺度、更高精度的地质勘探及其分析。基于多源异构数据的进一步融合，模型的预测精度及其可靠性将进一步得到提升，为服务深部资源开采、地下空间开发等复杂工程提供更加有科学依据的技术支撑。

从长远角度来看，本模型有望集成在智能化矿山综合管理平台上，从而形成集地质数据采集、三维建模以及稳定性分析与风险预警于一体的综合性决策支持系统，从而为实现矿山安全、高效以及绿色开发提供更为实时且精准的地质信息化服务。

参考文献

- [1] 刘灿灿, 郑西贵, 王术龙, 等, 基于钻孔成像的结构面智能化识别系统, 采矿与安全工程学报, 41(04):720-729, 2024.
- [2] 袁亮, 张平松, 煤矿透明地质模型动态重构的关键技术与路径思考, 煤炭学报, 48(01):1-14, 2023.
- [3] 康红普, 姜鹏飞, 王子越, 等, 煤巷钻锚一体化快速掘进技术与装备及应用, 煤炭学报, 49(01):131-151, 2024.
- [4] Wang C, Zou X, Han Z, et al., An automatic recognition and parameter extraction method for structural planes in borehole image, Journal of Applied Geophysics, 135:135-143, 2016.
- [5] 侯钦宽, 雍睿, 杜时贵, 等, 结构面粗糙度统计测量最小样本数确定方法, 岩土力学, 41(04):1259-1269, 2020.
- [6] 姚强岭, 李兴华, 朱柳, 等, 煤岩体地质力学参数原位测试系统开发与应用, 中国矿业大学学报, 48(06):1169-1176, 2019.
- [7] Jang H S, Kang S S, Jang B A, Determination of Joint Roughness Coefficients Using Roughness Parameters, Rock Mechanics and Rock Engineering, 47(6):2061-2073, 2014.
- [8] 杨凯, 林木景, 王晨龙, 等, 岩石节理轮廓线的数字表征方法影响研究, 太原理工大学学报, 52(05):797-802, 2021.
- [9] 孙辅庭, 余成学, 万利台, Barton 标准剖面 JRC 与独立于离散间距的统计参数关系研究, 岩石力学与工程学报, 33(S2):3539-3544, 2014.

附录 A 问题一主要代码（代码由 AI 辅助编程）

```
def canny_edge_detection(image_path, low_threshold=50,
high_threshold=150, use_morphology=True, kernel_size=1,
iterations=1):
    # 以下问题的部分代码由AI辅助编程
    """
    使用Canny算法进行边缘检测，并可选应用形态学操作（先膨胀后腐蚀）

    参数：
    image_path: 输入图像路径
    low_threshold: Canny低阈值
    high_threshold: Canny高阈值
    use_morphology: 是否使用形态学操作
    kernel_size: 形态学操作的核大小
    iterations: 形态学操作的迭代次数

    返回：
    原始Canny边缘检测结果和处理后的边缘检测结果（如果使用形态学操作）
    """
    # 读取图像
    image = cv2.imread(image_path)
    if image is None:
        raise ValueError(f"无法读取图像，请检查路径：{image_path}")
    print(f"成功读取图像：{image_path}，形状：{image.shape}")
    # 转换为灰度图
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    # 应用高斯模糊降噪
    blurred = cv2.GaussianBlur(gray, (5, 5), 1)
    # 使用Canny边缘检测
    canny_edges = cv2.Canny(blurred, low_threshold, high_threshold)
    print(f"Canny边缘检测后 - 非零像素数：
        {np.count_nonzero(canny_edges)}, 最大值：{np.max(canny_edges)}")
    # 如果需要，应用形态学操作（先膨胀后腐蚀）
    if use_morphology:
        # 创建形态学操作的核
        kernel = np.ones((kernel_size, kernel_size), np.uint8)
        print(f"应用形态学操作 - 核大小：{kernel_size}x{kernel_size},
            迭代次数：{iterations}")
```

```

# 先膨胀：连接断裂的边缘，填充小缺口
dilated = cv2.dilate(canny_edges, kernel,
                    iterations=iterations)
print(f"膨胀后 - 非零像素数: {np.count_nonzero(dilated)}, 最大值: {np.max(dilated)}")
# 再腐蚀：恢复边缘的原始宽度，去除小噪声
processed_edges = cv2.erode(dilated, kernel,
                           iterations=iterations)
print(f"腐蚀后 - 非零像素数: {np.count_nonzero(processed_edges)}, 最大值: {np.max(processed_edges)}")
return canny_edges, processed_edges
else:
    return canny_edges, canny_edges
def visualize_edges(original_path, canny_edges, processed_edges):

if __name__ == "__main__":
    image_path = "fujian3/output/3-3.jpg/step3_denoised.png"
    try:
        # 进行边缘检测，使用形态学操作（先膨胀后腐蚀）
        canny_edges, processed_edges =
            canny_edge_detection(image_path,
                                low_threshold=50,
                                high_threshold=100,
                                use_morphology=True,
                                kernel_size=0, # 使用最小的核大小
                                iterations=1) # 迭代次数

```

附录 B 问题二主要代码（代码由 AI 辅助编程）

```

class FractureClusterAnalyzer:
    # 以下问题的部分代码由AI辅助编程
    def __init__(self, image_folder):
        self.image_folder = image_folder
        self.results = {}

    def load_and_preprocess_images(self):
        """加载并预处理二值图像"""
        image_files = [f for f in os.listdir(self.image_folder)

```

```

        if f.endswith((''.png', '.jpg', '.jpeg', '.tif',
                        '.bmp'))]

self.images = {}
for img_file in image_files:
    img_path = os.path.join(self.image_folder, img_file)
    # 读取图像
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

    # 确保是二值图像：黑色裂隙(0)，白色背景(255)
    _, binary_img = cv2.threshold(img, 127, 255,
                                   cv2.THRESH_BINARY)

    self.images[img_file] = binary_img

return self.images

def extract_fracture_pixels(self, image):
    """提取裂隙像素的坐标"""
    # 找到黑色像素（裂隙）的位置
    fracture_coords = np.column_stack(np.where(image == 0))
    return fracture_coords

def extract_fracture_features(self, image):
    """提取裂隙的特征（位置和几何特征）"""
    # 方法1：直接使用像素坐标
    coords = self.extract_fracture_pixels(image)

    if len(coords) == 0:
        return np.array([])

    # 方法2：提取连通区域的特征
    labeled_image = measure.label(image == 0, connectivity=2)
    regions = measure.regionprops(labeled_image)

    features = []
    for region in regions:
        # 提取每个裂隙区域的多种特征
        centroid = region.centroid # 质心坐标

```

```

        area = region.area # 面积
        perimeter = region.perimeter # 周长
        eccentricity = region.eccentricity # 离心率
        bbox = region.bbox # 边界框

        # 组合特征
        feature_vector = [
            centroid[0], centroid[1], # 位置特征
            area, perimeter, eccentricity, # 形状特征
            bbox[2] - bbox[0], bbox[3] - bbox[1] # 尺寸特征
        ]
        features.append(feature_vector)

    return np.array(features)

def perform_gmm_clustering(self, features, n_components=3):
    """使用GMM进行聚类"""
    if len(features) == 0:
        return None, None

    # 确定最佳聚类数量 (如果未指定)
    if n_components is None:
        n_components = self.estimate_optimal_components(features)

    # 标准化特征
    from sklearn.preprocessing import StandardScaler
    scaler = StandardScaler()
    features_scaled = scaler.fit_transform(features)

    # 应用GMM聚类
    gmm = GaussianMixture(n_components=n_components,
                           random_state=42,
                           covariance_type='full')
    labels = gmm.fit_predict(features_scaled)

    return labels, gmm, scaler

def estimate_optimal_components(self, features,
                                max_components=10):

```

```

"""使用BIC准则估计最佳聚类数量"""
if len(features) < 2:
    return 1

bic_scores = []
n_components_range = range(1, min(max_components,
    len(features)) + 1)

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features)

for n_components in n_components_range:
    gmm = GaussianMixture(n_components=n_components,
        random_state=42)
    gmm.fit(features_scaled)
    bic_scores.append(gmm.bic(features_scaled))

# 选择BIC最小的聚类数
optimal_components = np.argmin(bic_scores) + 1
return optimal_components

def analyze_all_images(self, clustering_method='pixel',
    n_components=3):
    """分析所有图像"""
    self.load_and_preprocess_images()

    for img_name, image in self.images.items():
        print(f"处理图像: {img_name}")

        if clustering_method == 'pixel':
            # 基于像素坐标的聚类
            features = self.extract_fracture_pixels(image)
            if len(features) > 0:
                labels, gmm, scaler =
                    self.perform_gmm_clustering(features,
                        n_components)
            else:
                labels, gmm, scaler = None, None, None

```

```

elif clustering_method == 'region':
    # 基于区域特征的聚类
    features = self.extract_fracture_features(image)
    if len(features) > 0:
        labels, gmm, scaler =
            self.perform_gmm_clustering(features,
                                         n_components)
    else:
        labels, gmm, scaler = None, None, None

self.results[img_name] = {
    'image': image,
    'features': features,
    'labels': labels,
    'gmm': gmm,
    'scaler': scaler,
    'clustering_method': clustering_method
}

def visualize_results(self, save_dir=None,
color_scheme='viridis', show_colorbar=True):
    """可视化聚类结果 - 使用不同的颜色方案"""
    if not os.path.exists(save_dir) and save_dir:
        os.makedirs(save_dir)

    # 获取选定的颜色映射
    if color_scheme in self.color_schemes:
        cmap = self.color_schemes[color_scheme]
    else:
        cmap = plt.cm.viridis # 默认颜色方案
        print(f"颜色方案 {color_scheme} 不存在, 使用默认的 viridis 方案")

    for img_name, result in self.results.items():
        if result['labels'] is None:
            print(f"图像 {img_name} 无裂隙可聚类")
            continue

    fig, axes = plt.subplots(1, 2, figsize=(15, 6))

```

```

# 原始图像
axes[0].imshow(result['image'], cmap='gray')
axes[0].set_title(f'原始图像 - {img_name}')
axes[0].axis('off')

# 聚类结果
if result['clustering_method'] == 'pixel':
    # 像素级聚类可视化
    coords = result['features']
    labels = result['labels']

    scatter = axes[1].scatter(coords[:, 1], coords[:, 0],
                              c=labels, cmap=cmap, s=1, alpha=0.7)
    axes[1].set_title(f'GMM聚类结果 (像素级) - {color_scheme}')
    axes[1].invert_yaxis()
    axes[1].set_xlabel('X坐标')
    axes[1].set_ylabel('Y坐标')

elif result['clustering_method'] == 'region':
    # 区域级聚类可视化
    image = result['image'].copy()
    colored_image = np.stack([image] * 3, axis=-1)

    labeled_image = measure.label(image == 0,
                                   connectivity=2)
    regions = measure.regionprops(labeled_image)

    # 获取唯一的聚类标签
    unique_labels = np.unique(result['labels'])
    n_clusters = len(unique_labels)

    for i, region in enumerate(regions):
        if i < len(result['labels']):
            label = result['labels'][i]
            # 归一化标签到 [0,1] 范围用于颜色映射
            norm_label = label / max(1, n_clusters - 1)
            color = cmap(norm_label)

```

```

        # 为每个区域着色
        minr, minc, maxr, maxc = region.bbox
        mask = labeled_image[minr:maxr, minc:maxc] ==
            region.label
        colored_image[minr:maxr, minc:maxc][mask] =
            np.array(color[:3]) * 255

    axes[1].imshow(colored_image.astype(np.uint8))
    axes[1].set_title(f'GMM聚类结果 (区域级) - {color_scheme}')
    axes[1].axis('off')

    if show_colorbar:
        plt.colorbar(scatter, ax=axes[1], label='聚类标签')

def generate_cluster_report(self):
    """生成聚类分析报告"""
    report_data = []

    for img_name, result in self.results.items():
        if result['labels'] is not None and result['gmm'] is not
            None:
            n_clusters = result['gmm'].n_components
            labels = result['labels']

            cluster_info = {}
            for cluster_id in range(n_clusters):
                cluster_mask = (labels == cluster_id)
                cluster_size = np.sum(cluster_mask)
                cluster_info[f'Cluster_{cluster_id}_size'] =
                    cluster_size

            report_data.append({
                'Image': img_name,
                'Method': result['clustering_method'],
                'N_Clusters': n_clusters,
                **cluster_info,
                'Total_Fractures': len(labels)
            })

```

```

        report_df = pd.DataFrame(report_data)
        return report_df

# 使用示例
if __name__ == "__main__":
    # 初始化分析器
    analyzer = FractureClusterAnalyzer('inverted_masks')

    # 方法：基于像素坐标的聚类
    print("=== 基于像素坐标的GMM聚类 ===")
    analyzer.analyze_all_images(clustering_method='pixel',
                                n_components=3)

                                schemes=color_schemes)

    pixel_report = analyzer.generate_cluster_report()
    print(pixel_report)

```

附录 C 问题三主要代码（代码由 AI 辅助编程）

```

def calculate_z_stable(contour, min_dx=0.5):
    # 以下问题的部分代码由AI辅助编程
    """
    稳定计算Z值的方法
    min_dx: 最小x变化量阈值，避免除零和不稳定计算
    """
    points = contour[:, 0, :] # shape (N, 2)
    N = points.shape[0]
    if N < 2:
        return 0.0

    sum_slope_sq = 0.0
    valid_count = 0

    # 方法1：使用向量化的方式，过滤掉Δx太小的点
    for i in range(N - 1):
        dx = points[i+1, 0] - points[i, 0]
        dy = points[i+1, 1] - points[i, 1]

```

```

    # 只处理 $\Delta x$ 足够大的点对
    if abs(dx) > min_dx:
        slope = dy / dx
        sum_slope_sq += slope ** 2
        valid_count += 1

# 如果没有有效点对, 尝试方法2: 使用 $\Delta y / \Delta x$ 的替代方案
if valid_count == 0:
    for i in range(N - 1):
        dx = points[i+1, 0] - points[i, 0]
        dy = points[i+1, 1] - points[i, 1]

        # 如果 $\Delta x$ 太小但 $\Delta y$ 足够大, 使用大斜率值
        if abs(dy) > min_dx:
            if abs(dx) < 0.1: # 近似垂直
                slope_sq = 1e4 # 有限的大值, 而不是无穷大
            else:
                slope_sq = (dy/dx) ** 2
            sum_slope_sq += slope_sq
            valid_count += 1

if valid_count == 0:
    return 0.0

Z = math.sqrt(sum_slope_sq / valid_count)
return Z

def calculate_z_smooth(contour, window=6):
    """
    使用滑动平均平滑轮廓后计算Z值
    """
    points = contour[:, 0, :]
    N = points.shape[0]
    if N < 2:
        return 0.0

    # 对轮廓点进行平滑处理
    if N > window * 2:

```

```

    smoothed_points = []
    for i in range(N):
        start = max(0, i - window)
        end = min(N, i + window + 1)
        window_points = points[start:end]
        avg_x = np.mean(window_points[:, 0])
        avg_y = np.mean(window_points[:, 1])
        smoothed_points.append([avg_x, avg_y])
    points = np.array(smoothed_points)

sum_slope_sq = 0.0
valid_count = 0
epsilon = 1e-8

for i in range(0, N-1, 2): # 跳点采样, 减少噪声
    dx = points[i+1, 0] - points[i, 0]
    dy = points[i+1, 1] - points[i, 1]

    if abs(dx) > epsilon:
        slope = dy / dx
        sum_slope_sq += slope ** 2
        valid_count += 1

if valid_count == 0:
    return 0.0

Z = math.sqrt(sum_slope_sq / valid_count)
return min(Z, 1000) # 限制最大Z值

def calculate_jrc(Z):
    """根据Z计算JRC"""
    return 51.85 * (Z ** 0.6) - 10.37

def visualize_contours_corrected(image_path, output_folder,
    pixel_to_mm_ratio):
    """
    修正后的可视化函数
    """
    os.makedirs(output_folder, exist_ok=True)

```

```

img = cv2.imread(image_path)
if img is None:
    print(f"无法读取图像: {image_path}")
    return []

img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
img_contours = img.copy()

_, binary = cv2.threshold(img_gray, 127, 255, cv2.THRESH_BINARY)

# 使用轮廓近似来减少点数
contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL,
                                cv2.CHAIN_APPROX_NONE)

results = []
colors = [
    (255, 0, 0), (0, 255, 0), (0, 0, 255), (255, 255, 0),
    (255, 0, 255), (0, 255, 255), (255, 128, 0), (128, 0, 255),
]

print(f"物理尺寸转换参数: 1像素 = {pixel_to_mm_ratio:.6f} mm")

for idx, contour in enumerate(contours):
    if cv2.contourArea(contour) < 10:
        continue

    # 对轮廓进行近似, 减少点数
    epsilon = 0.01 * cv2.arcLength(contour, True)
    approx_contour = cv2.approxPolyDP(contour, epsilon, False)

    if len(approx_contour) < 2:
        continue

    # 计算物理长度
    contour_length_px = cv2.arcLength(contour, closed=False)
    contour_length_mm = contour_length_px * pixel_to_mm_ratio

    # 使用稳定方法计算Z值

```

```

Z1 = calculate_z_stable(contour)
Z2 = calculate_z_smooth(contour)

# 选择较小的Z值 (更合理)
Z = min(Z1, Z2)
JRC = calculate_jrc(Z)

results.append((Z, JRC, contour_length_mm))

color = colors[idx % len(colors)]
cv2.drawContours(img_contours, [contour], -1, color, 2)

return results

# 使用修正后的函数
if __name__ == "__main__":
    PIXEL_TO_MM = 0.38 # 你提供的参数

    folder_path = "test_binary"
    output_base_folder = "corrected_results"

    name_without_ext = os.path.splitext(filename)[0]
    output_folder = os.path.join(output_base_folder,
                                  name_without_ext)

    results = visualize_contours_corrected(image_path,
                                             output_folder, PIXEL_TO_MM)

```

附录 D 问题四主要代码 (代码由 AI 辅助编程)

```

BOREHOLE_PARAMS = [
    {"id": 1, "mouth": (500, 2000, 0), "bottom": (500, 2000,
    7000), "depth": 7000},
    {"id": 2, "mouth": (1500, 2000, 0), "bottom": (1500, 2000,
    7000), "depth": 7000},
    {"id": 3, "mouth": (2500, 2000, 0), "bottom": (2500, 2000,
    7000), "depth": 7000},

```

```

        {"id": 4, "mouth": (500, 1000, 0), "bottom": (500, 1000,
            5000), "depth": 5000},
        {"id": 5, "mouth": (1500, 1000, 0), "bottom": (1500, 1000,
            7000), "depth": 7000},
        {"id": 6, "mouth": (2500, 1000, 0), "bottom": (2500, 1000,
            7000), "depth": 7000}
    ]
    # 以下问题的部分代码由AI辅助编程
    DRILL_DIAMETER = 30
    DRILL_CIRCUMFERENCE = math.pi * DRILL_DIAMETER # 94.25mm(赛题问题)

    ADJACENT_PAIRS = [(1, 2), (1, 4), (2, 3), (2, 5), (3, 6), (4,
        5), (5, 6)] # 赛题相邻钻孔对

    GRID_SIZE_XY = 500 # X/Y轴网格尺度(汪进超文献5.2节)
    GRID_SIZE_Z = 100 # 修复: Z轴步长改为100mm, 覆盖裂隙深度(120-430mm)
    MAX_DEPTH = 7000

def load_prev_fissure_params():
    """
    固定前三问裂隙数据(基于《围岩裂隙精准识别与三维模型重构.pdf》问题1-3):
    每个钻孔2-3条有效裂隙, 参数符合赛题公式( $R=2.0-6\text{mm}$ 、 $P=94.25\text{mm}$ )与工程常识
    """
    裂隙数据fixed_fissure_data定义

    # 验证数据有效性(赛题裂隙定义:  $2R>1\text{mm}$ , 深度 $\leq 500\text{mm}$ )
    valid_fissures = {}
    for bh_id, fissures in fixed_fissure_data.items():
        valid = []
        for f in fissures:
            if 2 * f["R"] > 1 and f["depth_max"] <= 500:
                valid.append(f)
        valid_fissures[bh_id] = valid
    return valid_fissures

# 2.多钻孔分析核心类(基于指定文献)
class MultiBoreholeAnalyzer:
    def __init__(self):
        self.config = BoreholeConfig()

```

```

self.bh_params = self.config.BOREHOLE_PARAMS
# 直接加载固定数据
self.fissure_2d = load_prev_fissure_params()
self.fissure_3d = {} # 汪进超文献4.3节:三维平面二维→三维转换
self.conn_prob_matrix = {} # 赛题问题四:连通概率
self.uncertainty_data = None #
    汪进超文献5.2节:不确定性区域, 赛题:补充钻孔
self.supplement_boreholes = []

# 2.1 二维-三维平面转换(汪进超文献4.3节)
def convert_2d_to_3d_plane(self):
    """基于汪进超文献4.3节"孔壁轮廓曲线重构":由正弦参数推导三维平面方程"""
    for bh_id, fissures_2d in self.fissure_2d.items():
        bh = next(b for b in self.bh_params if b["id"] == bh_id)
        bh_mouth_x, bh_mouth_y, _ = bh["mouth"]
        fissures_3d = []

        for f in fissures_2d:
            R, beta, C, JRC = f["R"], f["beta"], f["C"], f["JRC"]
            f_depth_min, f_depth_max = f["depth_min"],
                f["depth_max"]

            # 倾向 $\alpha$ (汪进超文献方位角定义:正北为0, 顺时针为正)
            alpha = beta + 3 * math.pi / 2 if 0 <= beta < math.pi /
                2 else beta - math.pi / 2
            alpha %= 2 * math.pi

            # 倾角 $\gamma$ (赛题问题二隐含关系: $\gamma = \arctan(2R/\text{钻孔周长})$ )
            gamma = math.atan(2 * R /
                self.config.DRILL_CIRCUMFERENCE)

            # 三维平面法向量(A, B, C)(汪进超文献4.3节平面建模)
            A = math.sin(gamma) * math.cos(alpha)
            B = math.sin(gamma) * math.sin(alpha)
            C_plane = math.cos(gamma)

            # 平面截距 D(代入裂隙中心线坐标)
            D = -(A * bh_mouth_x + B * bh_mouth_y + C_plane * C)

```

```

# 裂隙中心点坐标
center_3d = (bh_mouth_x, bh_mouth_y, (f_depth_min +
    f_depth_max) / 2)

fissures_3d.append({
    "plane_params": (A, B, C_plane, D),
    "JRC": JRC,
    "depth_range": (f_depth_min, f_depth_max),
    "center_3d": center_3d
})

self.fissure_3d[bh_id] = fissures_3d

print(" 二维裂隙→三维平面转换完成(基于汪进超文献4.3节)")
return self.fissure_3d

# 2.2 连通概率计算(赛题+汪进超文献5.2节)
def calculate_connection_probability(self):
    """多因素加权:平面距离(0.4)、平面夹角(0.3)、JRC相似度(0.2)、深度重叠度(0.1)"""
    for (bh1_id, bh2_id) in self.config.ADJACENT_PAIRS:
        fissures_bh1 = self.fissure_3d.get(bh1_id, [])
        fissures_bh2 = self.fissure_3d.get(bh2_id, [])

        if not fissures_bh1 or not fissures_bh2:
            print(f"钻孔{bh1_id}-{bh2_id}无有效裂隙(赛题裂隙定义)")
            continue

        prob_matrix = np.zeros((len(fissures_bh1),
            len(fissures_bh2)))

        bh2 = next(b for b in self.bh_params if b["id"] == bh2_id)
        bh2_mouth_x, bh2_mouth_y, _ = bh2["mouth"]

        for i, f1 in enumerate(fissures_bh1):
            A1, B1, C1, D1 = f1["plane_params"]
            f1_jrc, (f1_zmin, f1_zmax) = f1["JRC"],
            f1["depth_range"]

            for j, f2 in enumerate(fissures_bh2):
                A2, B2, C2, D2 = f2["plane_params"]

```

```

f2_jrc, (f2_zmin, f2_zmax) = f2["JRC"],
    f2["depth_range"]

# 1.平面距离(汪进超文献4.2节公式)
numerator = abs(A1 * bh2_mouth_x + B1 * bh2_mouth_y
    + C1 * f2["center_3d"][2] + D1)
denominator = math.sqrt(A1**2 + B1**2 + C1**2)
dist = numerator / denominator if denominator != 0
    else 1000
distance_score = math.exp(-dist / 500)

# 2.平面夹角
dot_product = A1*A2 + B1*B2 + C1*C2
norm1, norm2 = math.sqrt(A1**2 + B1**2 + C1**2),
    math.sqrt(A2**2 + B2**2 + C2**2)
cos_theta = max(min(dot_product / (norm1 * norm2),
    1), -1)
angle_score = math.cos(math.acos(cos_theta) / 2)

# 3.JRC相似度(赛题问题三提示)
jrc_similarity = 1 - abs(f1_jrc - f2_jrc) / 20

# 4.深度重叠度
overlap = max(0, min(f1_zmax, f2_zmax) -
    max(f1_zmin, f2_zmin))
min_length = min(f1_zmax - f1_zmin, f2_zmax -
    f2_zmin)
overlap_score = overlap / min_length if min_length
    != 0 else 0

# 概率计算
total_score = 0.4 * distance_score + 0.3 *
    angle_score + 0.2 * jrc_similarity + 0.1 *
    overlap_score
prob = 1 / (1 + math.exp(-5 * (total_score - 0.5)))
prob_matrix[i, j] = round(prob, 3)

self.conn_prob_matrix[(bh1_id, bh2_id)] = prob_matrix

```

```

print(" 相邻钻孔裂隙连通概率计算完成(基于赛题与汪进超文献5.2节)")
return self.conn_prob_matrix

# 2.3 高不确定性区域识别(修复后, 汪进超文献5.2节)
def identify_high_uncertainty_regions(self):
    """
    修复点:1.Z轴网格步长改为100mm, 覆盖裂隙深度(120-430mm);
        2.增加索引有效性检查, 避免空索引循环;
        3.兜底处理空熵值数组, 符合工程常识。
    """
    # 生成网格(修复:Z轴步长100mm)
    x_range = np.arange(500, 2501, self.config.GRID_SIZE_XY)
    y_range = np.arange(1000, 2001, self.config.GRID_SIZE_XY)
    z_range = np.arange(0, self.config.MAX_DEPTH + 1,
                        self.config.GRID_SIZE_Z)

    grid_x, grid_y, grid_z = np.meshgrid(x_range, y_range,
                                           z_range, indexing="ij")
    entropy_grid = np.zeros(grid_x.shape)

    # 统计体素熵值(修复:增加索引有效性检查)
    for (bh1_id, bh2_id), prob_mat in
        self.conn_prob_matrix.items():
        fissures_bh1, fissures_bh2 = self.fissure_3d[bh1_id],
            self.fissure_3d[bh2_id]

        for i, f1 in enumerate(fissures_bh1):
            for j, f2 in enumerate(fissures_bh2):
                prob = prob_mat[i, j]
                if prob < 0.01:
                    continue

                # 裂隙对空间范围
                f1_x, f1_y, f1_z = f1["center_3d"]
                f2_x, f2_y, f2_z = f2["center_3d"]
                x_min, x_max = min(f1_x, f2_x), max(f1_x, f2_x)
                y_min, y_max = min(f1_y, f2_y), max(f1_y, f2_y)
                z_min, z_max = min(f1["depth_range"][0],
                                    f2["depth_range"][0]), max(f1["depth_range"][1],

```

```

        f2["depth_range"][1])

    # 匹配体素索引(修复:检查索引是否非空)
    x_idx = np.where((x_range >= x_min) & (x_range <=
        x_max))[0]
    y_idx = np.where((y_range >= y_min) & (y_range <=
        y_max))[0]
    z_idx = np.where((z_range >= z_min) & (z_range <=
        z_max))[0]

    if len(x_idx) == 0 or len(y_idx) == 0 or len(z_idx)
        == 0:
        continue # 跳过无匹配体素的裂隙对

    # 累加熵值
    for xi in x_idx:
        for yi in y_idx:
            for zi in z_idx:
                if prob > 0:
                    entropy_grid[xi, yi, zi] -= prob *
                        math.log2(prob)

    # 筛选高不确定性区域(修复:兜底处理空值数组)
    valid_entropy = entropy_grid[entropy_grid > 0]
    if len(valid_entropy) == 0:
        print("未统计到有效熵值, 使用默认值0.5(工程常识)")
        entropy_threshold = 0.5
        high_uncertainty_idx = np.where(entropy_grid >= 0) #
            取所有体素
    else:
        entropy_threshold = np.percentile(valid_entropy, 80) #
            80分位数
        high_uncertainty_idx = np.where(entropy_grid >=
            entropy_threshold)

    print(f"
        高不确定性区域识别完成:体素数{self.uncertainty_data['high_count']} (汪进超文献5
    return self.uncertainty_data

```

2.4 高不确定性区域可视化

```
def visualize_uncertainty_regions(self,
    save_path="uncertainty_regions_visualization.png"):
    """专门可视化高不确定性区域"""
    if self.uncertainty_data is None or
        len(self.uncertainty_data.get('high_x', [])) == 0:
        print("警告：无高不确定性区域数据可可视化")
        return

    high_x = self.uncertainty_data["high_x"]
    high_y = self.uncertainty_data["high_y"]
    high_z = self.uncertainty_data["high_z"]
    high_entropy = self.uncertainty_data["high_entropy"]

    # 创建3D图形
    fig = plt.figure(figsize=(15, 10))

    # 3D散点图
    ax1 = fig.add_subplot(121, projection='3d')

    # 绘制现有钻孔位置
    bh_colors = ["blue", "green", "red", "cyan", "magenta",
        "yellow"]
    for i, bh in enumerate(self.bh_params):
        x0, y0, z0 = bh["mouth"]
        ax1.plot([x0, x0], [y0, y0], [0, bh["depth"]],
            color=bh_colors[i], linewidth=6, alpha=0.8,
            label=f'钻孔{bh["id"]} ')
        ax1.scatter([x0], [y0], [0], color=bh_colors[i], s=100)

    ax1.set_xlabel('X轴 (mm)')
    ax1.set_ylabel('Y轴 (mm)')
    ax1.set_zlabel('Z轴 (mm)')
    ax1.set_title('高不确定性区域3D分布\n(颜色越暖表示不确定性越高)')
    ax1.legend(loc='upper left', bbox_to_anchor=(0, 1))

    # 2D平面投影图
    ax2 = fig.add_subplot(122)
```

```

# 按深度分层显示
unique_depths = np.unique(high_z)
depth_colors = plt.cm.viridis(np.linspace(0, 1,
    len(unique_depths)))

for i, depth in enumerate(unique_depths):
    mask = high_z == depth
    if np.any(mask):
        ax2.scatter(high_x[mask], high_y[mask],
            c=[depth_colors[i]], s=50 +
                (high_entropy[mask] / np.max(high_entropy))
                * 100,
            alpha=0.7, label=f'深度 {depth}mm')

# 绘制钻孔位置
for i, bh in enumerate(self.bh_params):
    x0, y0, _ = bh["mouth"]
    ax2.scatter(x0, y0, color=bh_colors[i], s=150, marker='s',
        label=f'钻孔{bh["id"]}', edgecolors='black')

ax2.set_xlabel('X轴 (mm)')
ax2.set_ylabel('Y轴 (mm)')
ax2.set_title('高不确定性区域平面投影\n(按深度分层)')
ax2.grid(True, alpha=0.3)
ax2.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
plt.savefig(save_path, dpi=300, bbox_inches='tight')
plt.show()

print(f" 高不确定性区域可视化完成, 保存至: {save_path}")

# 打印统计信息
print(f"\n高不确定性区域统计:")
print(f"- 总高不确定性体素数: {len(high_x)}")
print(f"- 熵值范围: {np.min(high_entropy):.3f} -
    {np.max(high_entropy):.3f}")
print(f"- 平均熵值: {np.mean(high_entropy):.3f}")

```

```

print(f"- 涉及深度层数: {len(unique_depths)}")
print(f"- 深度范围: {np.min(high_z)} - {np.max(high_z)} mm")

return fig

# 2.5 补充钻孔优化
def optimize_supplement_boreholes(self, top_k=3):
    """约束:与现有钻孔间距≥500mm、孔深5000mm; 目标:信息增益最大化"""

    if self.uncertainty_data is None or
       len(self.uncertainty_data.get('high_x', [])) == 0:
        print("警告: 高不确定性区域为空, 使用备选方案生成补充钻孔")
        return self._generate_fallback_supplement_boreholes(top_k)

    high_x = self.uncertainty_data["high_x"]
    high_y = self.uncertainty_data["high_y"]
    existing_mouths = {(b["mouth"][0], b["mouth"][1]) for b in
                       self.bh_params}

    print(f"高不确定性区域坐标数量: {len(high_x)}")

    # 生成候选位置
    candidates = []
    candidate_coords = set()

    for x, y in zip(high_x, high_y):
        if any(abs(x - cx) < 200 and abs(y - cy) < 200 for cx, cy
              in candidate_coords):
            continue

        min_dist = min(distance.euclidean((x, y), (mx, my)) for
                       mx, my in existing_mouths)

        if min_dist >= 300:
            candidates.append({
                "x": round(x, 0),
                "y": round(y, 0),
                "z": 0,
                "depth": 5000,
            })

```

```

        "info_gain": 0,
        "min_dist": min_dist,
        "source": "high_uncertainty"
    })
    candidate_coords.add((x, y))

# 如果候选不足, 生成中间位置候选点
if len(candidates) < top_k:
    mid_candidates =
        self._generate_midpoint_candidates(existing_mouths,
            candidate_coords)
    candidates.extend(mid_candidates)

# 计算信息增益
if len(candidates) > 0:
    grid_x = self.uncertainty_data["grid_x"]
    grid_y = self.uncertainty_data["grid_y"]
    entropy_grid = self.uncertainty_data["entropy"]

    for cand in candidates:
        cand_x, cand_y = cand["x"], cand["y"]
        x_idx = np.argmin(np.abs(grid_x[:, 0, 0] - cand_x))
        y_idx = np.argmin(np.abs(grid_y[0, :, 0] - cand_y))

        x_start, x_end = max(0, x_idx-1), min(grid_x.shape[0],
            x_idx+2)
        y_start, y_end = max(0, y_idx-1), min(grid_y.shape[1],
            y_idx+2)

        region_entropy = entropy_grid[x_start:x_end,
            y_start:y_end, :]
        if region_entropy.size > 0 and np.any(region_entropy >
            0):
            cand["info_gain"] =
                round(np.mean(region_entropy[region_entropy >
                    0]), 3)
        else:
            cand["info_gain"] = 0.5

```

```

# 选择最终候选点
candidates_sorted = sorted(candidates, key=lambda x:
    x["info_gain"], reverse=True)
final_candidates = []
used_regions = set()

for cand in candidates_sorted:
    region_key = (round(cand["x"] / 500), round(cand["y"] /
        500))
    if region_key not in used_regions:
        final_candidates.append(cand)
        used_regions.add(region_key)
    if len(final_candidates) >= top_k:
        break

if len(final_candidates) < top_k:
    for cand in candidates_sorted:
        if cand not in final_candidates:
            final_candidates.append(cand)
        if len(final_candidates) >= top_k:
            break

self.supplement_boreholes = [
    {
        "优先级": i + 1,
        "孔口坐标(X, Y, Z)": (c["x"], c["y"], c["z"]),
        "孔深(mm)": c["depth"],
        "信息增益": c["info_gain"],
        "最小间距(mm)": c["min_dist"],
        "来源": c.get("source", "中间位置")
    }
    for i, c in enumerate(final_candidates[:top_k])
]

print(" 补充钻孔位置确定完成")
return self.supplement_boreholes

def _generate_midpoint_candidates(self, existing_mouths,
    exclude_coords):

```

```

"""生成现有钻孔中间位置的候选点"""
candidates = []
existing_list = list(existing_mouths)
xs = [p[0] for p in existing_list]
ys = [p[1] for p in existing_list]
center_x, center_y = np.mean(xs), np.mean(ys)

grid_points = []
for dx in [-800, -400, 0, 400, 800]:
    for dy in [-800, -400, 0, 400, 800]:
        x = center_x + dx
        y = center_y + dy
        if (500 <= x <= 2500 and 1000 <= y <= 2000 and
            not any(abs(x - cx) < 200 and abs(y - cy) < 200
                    for cx, cy in exclude_coords)):
            grid_points.append((x, y))

for x, y in grid_points:
    min_dist = min(distance.euclidean((x, y), (mx, my)) for
                    mx, my in existing_mouths)
    if min_dist >= 300:
        candidates.append({
            "x": round(x, 0),
            "y": round(y, 0),
            "z": 0,
            "depth": 5000,
            "info_gain": 0.6,
            "min_dist": min_dist,
            "source": "midpoint"
        })

return candidates

def _generate_fallback_supplement_boreholes(self, top_k=3):
    """备选方案生成补充钻孔"""
    existing_mouths = {(b["mouth"][0], b["mouth"][1]) for b in
                        self.bh_params}
    xs = [p[0] for p in existing_mouths]
    ys = [p[1] for p in existing_mouths]

```

```

center_x, center_y = np.mean(xs), np.mean(ys)

fallback_positions = [
    (center_x + 600, center_y + 600),
    (center_x - 600, center_y + 600),
    (center_x + 600, center_y - 600)
]

candidates = []
for x, y in fallback_positions:
    if 500 <= x <= 2500 and 1000 <= y <= 2000:
        min_dist = min(distance.euclidean((x, y), (mx, my)) for
            mx, my in existing_mouths)
        if min_dist >= 300:
            candidates.append({
                "x": round(x, 0),
                "y": round(y, 0),
                "z": 0,
                "depth": 5000,
                "info_gain": 0.7,
                "min_dist": min_dist,
                "source": "fallback"
            })

self.supplement_boreholes = [
    {
        "优先级": i + 1,
        "孔口坐标(X, Y, Z)": (c["x"], c["y"], c["z"]),
        "孔深(mm)": c["depth"],
        "信息增益": c["info_gain"],
        "最小间距(mm)": c["min_dist"],
        "来源": c["source"]
    }
    for i, c in enumerate(candidates[:top_k])
]

return self.supplement_boreholes

```

3.主函数

```
def main():  
    try:  
        analyzer = MultiBoreholeAnalyzer()  
        analyzer.convert_2d_to_3d_plane()  
        analyzer.calculate_connection_probability()  
        analyzer.identify_high_uncertainty_regions()
```